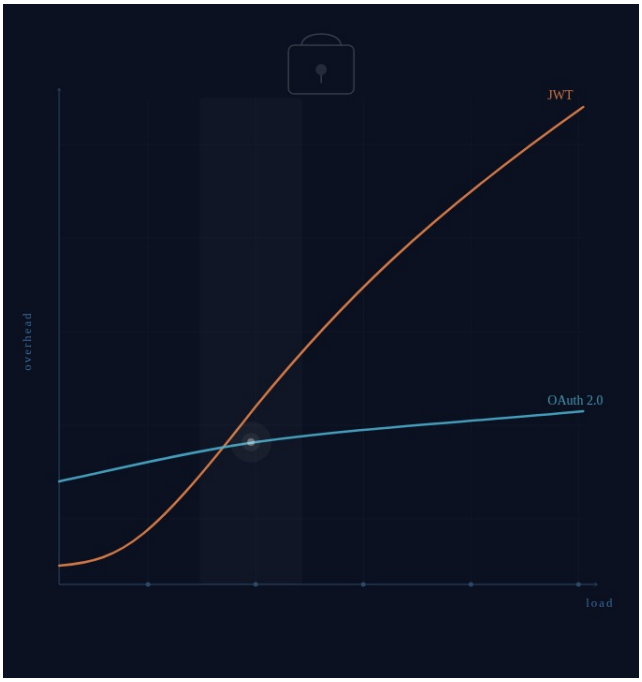
DEGREE PROJECT

A Quantitative Analysis of the Performance Overhead of API Security Solutions in Simulated SME Environments



Jonathan Zulu

Information Security, master's level (120 credits)
2026

Luleå University of Technology
Department of Computer Science, Electrical and Space Engineering

LULEÅ UNIVERSITY OF TECHNOLOGY

[This page intentionally left blank]

A Quantitative Analysis of the Performance Overhead of API Security Solutions in Simulated SME Environments

Master's Thesis in Information Security

Jonathan Zulu

jonzul-1@student.ltu.se

Supervisor: Prof. Tero Päivärinta

Programme: Master's in Information Security

Department: Computer Science and Electrical Engineering

University: Luleå University of Technology

Date: June 2026

Abstract

Small and medium enterprises (SMEs) increasingly depend on Application Programming Interfaces (APIs) to run their services, and securing those APIs imposes a performance price. Every authentication check, verifying that an incoming request is legitimate before letting it through, consumes computing time that could otherwise go toward serving the user. For a large company with powerful servers, this cost is negligible. For an SME running its services on a single constrained server, it may not be — yet no controlled evidence exists on how the common API security patterns actually perform under such constraints.

This thesis asks a simple question with a counter-intuitive answer: which API security approach costs the least in performance terms, and does the answer change as traffic grows? Three common approaches were compared along two architectural dimensions: a stateless, CPU-bound pattern (JSON Web Token authentication), a stateful, I/O-bound delegated authorisation pattern (OAuth 2.0 token verification), and a stateful, I/O-bound traffic control pattern (Redis-backed rate limiting), alongside an unsecured baseline. Each was tested in a Docker container restricted to the kind of hardware a typical SME might use: one CPU core and one gigabyte of memory. Load was varied across five levels, from a quiet 15 concurrent users to a busy 600, with five repeated measurements per condition across 100 experimental runs in total.

The central finding is a reversal that contradicts the conventional assumption that stateless authentication is universally faster than delegated alternatives. The stateless CPU-bound pattern JWT, which is widely regarded as the faster option due to its stateless design, was indeed fastest at low traffic, adding less than a millisecond of overhead at 15 concurrent users. But as traffic increased to 75 users and beyond, the CPU-bound verification serialised under single-core saturation and JWT became the slowest of the three, eventually adding 116.84 milliseconds of overhead at 600 users. The two I/O-bound patterns: OAuth 2.0 and rate limiting, whose fixed verification cost amortises rather than compounds under concurrent load, by contrast added no more than roughly 18 milliseconds across the same range. Security pattern, load level, and their interaction each produced statistically significant effects with very large effect sizes ($\eta_p^2 > .98$ for all three).

This finding is characterised as the Load-Dependent Authentication Overhead Crossover (LDAOC): an empirically grounded extension of Security-Performance Tradeoff Theory demonstrating that authentication overhead under resource constraints is a dynamic, load-dependent function rather than a fixed additive cost. The 4×5 factorial measurement framework employed here is designed for transferability and invites independent verification across alternative runtimes, hardware tiers, and deployment topologies.

The practical implication is direct. An SME that benchmarks its API at low traffic and selects a CPU-bound stateless authentication pattern such as JWT on performance grounds will find that choice working against it once user numbers grow, precisely when perfor-

mance matters most. A load-stratified decision framework is proposed to help practitioners match their authentication choice, and its underlying architectural characteristics, to their actual operating conditions rather than their development environment.

Keywords: API security, performance overhead, JWT authentication, OAuth 2.0, small and medium enterprises, resource constraints, authentication crossover, controlled experiment

Contents

1 Introduction 1

1.1 Background 1

1.2 Problem Statement 2

1.3 Research Purpose and Questions 2

1.4 Research Approach 3

1.5 Scope and Delimitations 3

1.6 Expected Contributions 4

1.7 Thesis Structure 4

2 Literature Review and Theoretical Framework 6

2.1 Key Concept Definitions and Seminal Literature 7

2.1.1 API Security Architectural Pattern 7

2.1.2 Performance Overhead 9

2.1.3 High-Tech SME Context 9

2.1.4 REST Architectural Style 10

2.2 API Security Landscape 10

2.2.1 API Threat Environment 11

2.2.2 API Security Solution Categories 11

2.3 Performance Engineering Foundations 12

2.3.1 Software Performance Engineering Principles 12

2.3.2 Security-Performance Trade-off Literature 13

2.3.3 Queueing Theory Foundations 14

2.4 SME-Specific Context and Constraints 14

2.4.1 Resource Constraints and Their Implications 14

2.4.2 SME Security Decision-Making Behaviour 15

2.5 State of the Art: API Security Performance 16

2.6 Theoretical Framework 17

2.6.1 REST Architectural Style (Type I + Type V) 17

2.6.2 Security-Performance Trade-off Theory (Type III–IV) 18

2.6.3 Resource-Based SME Constraint Theory (Type IV) 18

2.6.4 Software Performance Engineering (Type V) 19

2.6.5 Complementary Theories 19

2.6.6 Theory Integration and Research Model 20
2.7 Research Gap and Expected Contribution 22
3 Research Methodology 23
3.1 Research Design 23
3.2 Conceptualisation of Variables 23
3.2.1 Independent Variable: Authentication Architectural Pattern 23
3.2.2 Dependent Variables 24
3.2.3 Control Variables 25
3.3 Experimental Infrastructure 25
3.3.1 Hardware and Network Configuration 25
3.3.2 Security Pattern Implementations 25
3.3.3 Containerisation and Orchestration 26
3.4 Experimental Design and Procedure 26
3.4.1 Factorial Design 26
3.4.2 Load Test Configuration 27
3.4.3 Execution Protocol 27
3.4.4 Research Hypotheses 27
3.5 Quantitative Data Analysis 30
3.5.1 Overhead Computation 30
3.5.2 Statistical Analysis 30
3.5.3 SME Acceptability Threshold 30
3.6 Ethical Considerations 31
3.7 Validity Considerations 31
3.7.1 Internal Validity 31
3.7.2 External Validity 31
3.7.3 Construct Validity 31
3.7.4 Reliability 32
3.8 Summary 32
4 Results and Analysis 33
4.1 Descriptive Statistics and Interaction Overview 33
4.2 H1–H3: Two-Way Factorial ANOVA 35
4.2.1 ANOVA Assumption Checks 35
4.2.2 Omnibus ANOVA Results 36
4.2.3 Post-Hoc Pairwise Comparisons 37
4.3 RQ1: Authentication Overhead Profiles Under SME Resource Constraints 38
4.3.1 JWT Overhead Behaviour 38
4.3.2 OAuth2 Overhead Behaviour 38
4.3.3 Rate-Limiting Overhead Behaviour 38

4.4 RQ2: Throughput Impact . . . 39
4.5 H4: Practical Threshold Analysis (RQ3) . . . 39
4.6 Crossover Analysis: Load-Dependent Performance Inversion . . . 40
4.6.1 Mechanistic Explanation . . . 40
4.6.2 Implications for RQ1 and RQ2 . . . 41
4.7 Summary of Findings . . . 41
5 Discussion . . . 43
5.1 Hypothesis Verdicts and Interpretation Against Theory . . . 43
5.1.1 H1: Security Pattern Affects Response Time (Supported) . . . 43
5.1.2 H2: Interaction Between Security Pattern and Load Level (Supported) . . . 44
5.1.3 H3: Load Level Affects Response Time (Supported) . . . 45
5.1.4 H4: Practitioner Threshold Exceedance (Supported) . . . 45
5.2 The Load-Dependent Authentication Overhead Crossover: An Empirical Extension of Security-Performance Trade-off Theory . . . 46
5.2.1 Characterisation and Definition . . . 46
5.2.2 Positioning Within Existing Theory . . . 47
5.2.3 Scope Conditions . . . 47
5.3 Practical Decision Framework for SMEs . . . 48
5.3.1 Load-Stratified Selection Guidance . . . 48
5.3.2 Summary of Recommendations . . . 49
5.3.3 Broader Implication for SME Security Decision-Making . . . 49
5.3.4 Application to Hybrid Architectures . . . 50
5.3.5 Security Adequacy and Operational Cost Implications . . . 51
5.4 Limitations and Alternative Explanations . . . 51
5.4.1 Single-Core CPU Constraint . . . 52
5.4.2 Node.js Single-Threaded Event Loop . . . 52
5.4.3 Synthetic OAuth 2.0 Introspection Delay . . . 53
5.4.4 Co-Location of Load Generator and APIs . . . 53
5.4.5 Synthetic Workload . . . 54
5.4.6 Absence of Token Lifecycle Effects . . . 54
5.5 Generalisability . . . 54
5.5.1 What is Likely Generalisable . . . 54
5.5.2 What Requires Independent Verification . . . 55
5.5.3 What Should Not Be Generalised . . . 55
5.6 Future Research Directions . . . 56
5.7 Summary . . . 56
6 Conclusion . . . 58
6.1 Answering the Research Questions . . . 58

6.2 Summary of Contributions 59
6.2.1 Empirical Contribution 59
6.2.2 Theoretical Contribution 59
6.2.3 Practical Contribution 59
6.3 Limitations 59
6.4 Future Work 60
6.5 Closing Statement 60

A Experimental Configuration 61
A.1 Docker Compose Configuration 61
A.2 API Implementation 62
A.3 Locust Load Test Configuration 63
A.4 Master Experiment Script 63

B Complete Statistical Output 65
B.1 Shapiro-Wilk Normality Tests (Per Cell) 65
B.2 Levene's Test for Homogeneity of Variance 65
B.3 Two-Way Factorial ANOVA (Full Output) 66
B.4 Post-Hoc Pairwise Comparisons 66
B.5 Kruskal-Wallis Sensitivity Check 66

C Raw Experimental Data 68

D Additional Figures 70
D.1 Throughput by Security Pattern and Load Level 70
D.2 Response Time Distribution (Box Plots) 70
D.3 Overhead Percentage (Relative) by Load Level 71

Chapter 1

Introduction

1.1 Background

Modern businesses, from online retailers to software-as-a-service providers, increasingly depend on Application Programming Interfaces (APIs) to connect their systems, serve their customers, and integrate with third-party services [1]. Every time a mobile app loads a product catalogue, a payment is confirmed, or an internal dashboard refreshes, an API request is made and processed. Securing these requests, verifying that each one comes from a legitimate source and is authorised to access the requested resource, is a fundamental requirement. Without effective API security, businesses expose themselves to data breaches, service disruption, and financial loss.

The security measures that protect APIs are not free. Each authentication check, each token verification, each rate-limiting lookup consumes computational resources: CPU cycles, memory, and time. On a well-provisioned enterprise server with multiple processor cores and abundant memory, this cost is negligible. But Small and Medium Enterprises (SMEs) rarely operate such infrastructure. Cost-conscious and resource-constrained, SMEs typically host their services on minimal cloud instances, often a single virtual CPU with one gigabyte of memory [2]. On this hardware, the computational cost of security is no longer negligible; it can become the factor that determines whether an API responds quickly enough to keep customers engaged or slowly enough to drive them away.

This creates a dilemma. SMEs need API security just as much as large enterprises, arguably more, given that a single breach can be proportionally more damaging to a smaller organisation [3]. The European Union Agency for Cybersecurity confirms that SMEs face distinct challenges requiring tailored approaches, as generic enterprise frameworks impose disproportionate complexity on organisations with limited technical staff [4, 5]. Yet the security guidance available to them was largely developed for and tested on enterprise-grade infrastructure. The OWASP API Security Top 10 [6] tells practitioners *what* threats to defend against but not *how much* that defence will cost in terms of system performance. Academic research has studied authentication overhead, but mostly under conditions where computational resources are abundant and the performance cost is a rounding error [7, 8]. For an SME operating at the margins of its hardware capacity, this is not a minor gap, it is the difference between following advice that works and following advice that inadvertently

makes the service slower precisely when it matters most.

1.2 Problem Statement

The core problem this thesis addresses is the absence of empirical evidence on how API security mechanisms perform under the resource constraints that SMEs actually experience. Two widely adopted authentication approaches, JSON Web Tokens (JWT) and OAuth2, represent fundamentally different architectural strategies [9, 10]. JWT verifies each request by performing a cryptographic signature check locally, consuming CPU on every call [11, 12]. OAuth2 delegates verification to an external authorisation server, consuming network I/O on every call [13]. A third common pattern, rate limiting backed by an in-memory data store such as Redis, protects against excessive request volumes through fast key-value lookups.

Conventional wisdom, reinforced by practitioner guides and framework documentation, treats JWT as the faster option because it avoids external network calls. This assumption is reasonable when CPU capacity is abundant. But on a single-core server approaching saturation, the dynamics may be different: a mechanism that is CPU-bound (JWT) competes for the same constrained resource that the application itself needs, while a mechanism that is I/O-bound (OAuth2) offloads work to a different resource pathway. Whether this difference is practically significant, and at what traffic level it becomes consequential, has not been empirically measured under SME-realistic conditions.

Without this evidence, SME practitioners make security architecture decisions based on assumptions derived from enterprise contexts [14], potentially selecting mechanisms that perform well at low traffic but degrade disproportionately as their businesses grow.

1.3 Research Purpose and Questions

The purpose of this research is to provide empirical evidence on the performance overhead of API security architectural patterns under resource constraints representative of SME deployments, enabling evidence-based security architecture decisions where currently only assumption-based guidance exists.

Three research questions guide the investigation:

RQ1. What is the measurable performance overhead introduced by different API authentication mechanisms under SME-realistic resource constraints?

RQ2. How does the interaction between authentication architectural pattern and concurrent user load affect API response time and throughput?

RQ3. Under what conditions does authentication overhead remain within a practically acceptable threshold for SME-hosted API deployments?

RQ1 establishes the baseline facts: how much overhead does each pattern add? RQ2 probes whether the relationship between pattern and overhead changes as load increases, that is, whether the patterns scale differently. RQ3 translates the statistical findings into practical guidance by assessing which patterns remain within a performance budget that practitioners consider acceptable.

1.4 Research Approach

The SMEs of interest here are *high-tech* SMEs: organisations with fewer than 250 employees [15] that develop or depend heavily on software and for which API infrastructure is central to revenue generation. Such organisations possess real technical capability but remain resource-constrained, commonly hosting production services on a single minimal cloud instance rather than on horizontally scaled infrastructure [2]. This operational profile is what the experimental setting reproduces: each API is deployed in a container limited to 1 vCPU and 1 GB RAM (detailed in Section 3.3), so that the measured overhead reflects the conditions these SMEs actually face rather than enterprise-grade abundance.

The research employs a controlled laboratory experiment following established practices in empirical software engineering [16]. Three API security architectural patterns (JWT, OAuth2, and Redis-backed rate limiting) plus an unsecured baseline are tested across five concurrent user load levels (15, 75, 150, 300, and 600 users) in a 4×5 factorial design with five replications per condition, totalling 100 experimental runs. The unsecured baseline is included as the fourth level of the Security Pattern factor, enabling the ANOVA to correctly partition variance attributable to authentication overhead against the no-auth condition. Each API implementation is deployed in a Docker container [17, 18] constrained to 1 vCPU and 1 GB RAM, simulating the entry-level cloud infrastructure commonly used by SMEs.

Authentication overhead is measured as the difference in mean response time between each secured pattern and the unsecured baseline at the same load level. A two-way factorial ANOVA tests for main effects of security pattern and load level, as well as their interaction. A literature-grounded 10 ms threshold provides a practical significance criterion against which each condition is assessed.

1.5 Scope and Delimitations

The study is deliberately narrow in scope to enable precise causal claims. The following boundaries apply:

- **Hardware**: A single physical server hosting Docker containers, each constrained to 1 vCPU and 1 GB RAM. Multi-core and horizontally scaled deployments are outside the scope.
- **Security patterns**: JWT (HMAC-SHA256), OAuth2 (token introspection with a synthetic 5 ms LAN-adjacent delay), and Redis-backed sliding window rate limiting. Centralised API gateway solutions (e.g., Kong) were excluded from the empirical scope due to methodological constraints and are identified as future work.
- **Implementation stack**: Node.js 18.x with Express. Findings are specific to this stack; generalisation to other runtimes requires independent replication.
- **Workload**: Stateless, read-heavy API requests with identical synthetic business logic across all patterns. Real-world workload heterogeneity is not captured.
- **Network**: Co-located containers on a single Docker host with sub-millisecond inter-container latency. Distributed deployments with real network variability are outside the scope.

These delimitations are not limitations in the pejorative sense; they are the design choices that make the experimental results internally valid. Relaxing any one of them would require a separate study, and each is identified as a future research direction in Chapter 5.

1.6 Expected Contributions

The research contributes across three domains:

Empirical contribution. The first controlled comparison of API security pattern performance under SME-realistic resource constraints, providing overhead measurements across multiple load conditions that did not previously exist in the literature.

Theoretical contribution. An empirically grounded extension of Security-Performance Trade-off Theory [19], demonstrating that authentication overhead under resource constraints is a dynamic, load-dependent function rather than a fixed additive cost, a qualification not captured by models developed in resource-abundant environments.

Practical contribution. A load-stratified decision framework enabling SME practitioners to select authentication patterns based on expected traffic, replacing assumption-based defaults with evidence-based guidance.

1.7 Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2 presents the literature review and theoretical framework, establishing key concept definitions, surveying the state of the art in API security performance, and developing the integrated theoretical model that guides the research.

Chapter 3 describes the research methodology, including the factorial experimental design, variable operationalisation, infrastructure configuration, and validity considerations.

Chapter 4 reports the experimental findings, structured around the four hypotheses derived from the research questions, with descriptive statistics, ANOVA results, and post-hoc comparisons.

Chapter 5 interprets the findings in relation to the theoretical framework, characterises the observed performance crossover phenomenon, develops the practitioner decision framework, and critically examines limitations and generalisability.

Chapter 6 synthesises the contributions, states the conclusions, and identifies directions for future research.

Chapter 2

Literature Review and Theoretical Framework

This chapter reviews the literature relevant to understanding the performance implications of API security architectural patterns in small and medium enterprise (SME) environments. Following Webster and Watson's [20] concept-centric approach, complemented by the systematic-review guidelines of Kitchenham et al. [21], it organises the literature around key research dimensions rather than chronologically or by author. The chapter establishes key concept definitions grounded in seminal academic sources, surveys the current state of knowledge across API security, performance engineering, and SME-specific constraints, and presents the theoretical framework guiding this research. The chapter concludes by synthesising the identified research gap that motivates the empirical study.

Table 2.1 maps the reviewed literature against the principal research dimensions, providing an orientation to the state of the art before the detailed review. The matrix is restricted to sources providing direct empirical or technical evidence on API security performance; theoretical, methodological, and organisational sources are cited in the relevant sections but excluded from this state-of-the-art mapping. Column themes are aligned with the architectural dimensions introduced in Section 2.1.1: authentication mechanism complexity, security function deployment pattern, and traffic control mechanism. The matrix reveals strong coverage of authentication mechanism specifications and SME vulnerability context, moderate coverage of security function deployment and traffic control, and a notable absence of sources addressing the intersection of security performance with SME-specific resource constraints, precisely the gap this research targets.

Table 2.1: Concept Matrix, API Security Performance Literature (•• = primary focus; • = addressed; – = not addressed). Columns aligned to Section 2.1.1 architectural dimensions. Theoretical and methodological sources cited in text but excluded from this empirical SotA mapping.

#	Source	*Auth Mech. Complexity*	*Sec. Function Deployment*	*Traffic Control*	*Perf. Overhead Data*	*Operational Scope*	*RQ Relevance*
1	RFC 6749; RFC 7519	••	–	–	– (spec)	•	••
2	OWASP (2023)	•	•	••	–	•	••
3	Sun et al. (2022)	••	•	–	–	•	•
4	Yarygina & Bagge (2018)	•	••	–	••	–	••
5	Chatterjee & Prinz (2022)	••	•	–	••	–	••
6	Berardi et al. (2022)	•	••	•	–	–	•
7	Akamai (2024); Salt (2024)	•	–	•	–	•	•
8	Eling & Wirfs (2019)	–	–	–	–	••	•
9	Barlette & Fomin (2008)	–	–	–	–	••	•
10	Chidukwani et al. (2022)	–	–	–	–	••	•
11	ENISA (2021)	–	–	–	–	••	•
12	Wolter & Reinecke (2010)	•	–	–	••	–	••
13	Felter et al. (2015)	–	–	–	••	–	•
14	Jiang & Hassan (2015)	–	–	•	••	–	•
	Sources per dimension	**8**	**5**	**4**	**5**	**8**	

2.1 Key Concept Definitions and Seminal Literature

This section establishes precise definitions for the core concepts used throughout this thesis. Each definition is grounded in seminal academic sources to ensure conceptual clarity and consistency.

2.1.1 API Security Architectural Pattern

The independent variable in this research is the API security architectural pattern, defined as the architectural approach to implementing API protection, characterised by underlying conceptual dimensions rather than specific technology implementations. This conceptualisation addresses the distinction between technology names and the theoretical constructs they represent. Specific technologies such as JWT, OAuth2, and rate limiting mechanisms are operationalisations of positions along conceptual dimensions, not variables in them-

selves.

Three conceptual dimensions characterise API security architectural patterns. The first dimension, **authentication mechanism complexity**, captures the degree to which authentication requires external coordination versus self-contained validation. This dimension ranges from stateless token validation, where the resource server independently verifies credentials using locally available information, to delegated authorisation, where authentication requires communication with an external authorisation server. This dimension is grounded in Fielding's [22] statelessness constraint, which stipulates that each request from client to server must contain all information necessary to understand the request. JWT (RFC 7519) [11] operationalises stateless authentication through self-contained tokens carrying embedded claims verified through cryptographic signatures. OAuth2 (RFC 6749) [13] operationalises delegated authorisation through token introspection requiring external server communication. These two patterns represent fundamentally different positions within the broader taxonomy of authentication mechanisms [23].

The second dimension, **security function deployment pattern**, captures whether security functions are distributed across individual service components or centralised through a dedicated gateway component. This dimension maps to Fielding's [22] layered system constraint, which permits architectural components to be organised in hierarchical layers with each layer only aware of adjacent layers. Distributed deployment places security logic within each service, while centralised deployment routes all traffic through a gateway that enforces security policies before requests reach backend services. The distinction has direct performance implications: centralised deployment introduces an additional network hop but enables consolidated policy enforcement, while distributed deployment avoids this intermediary but may introduce redundant processing across services. The centralised gateway deployment pattern falls outside the empirical scope of this study but is identified as a priority direction for future research (Chapter 5).

The third dimension, **traffic control mechanism**, captures the presence or absence of request rate management at the architectural level. Unlike authentication and authorisation, which verify identity and permissions, traffic control mechanisms such as rate limiting address availability protection by restricting the volume of requests a client can submit within a defined time window. This dimension is architecturally distinct because it operates on request metadata (source IP, API key, request count) rather than identity claims, introducing a different category of processing overhead. The OWASP API Security Top 10 [6] identifies unrestricted resource consumption as a critical vulnerability category, establishing traffic control as a fundamental security concern separate from authentication.

This three-dimensional conceptualisation enables the research to examine which architectural characteristics drive performance differences, rather than merely comparing technology labels. The three security patterns studied empirically (JWT, OAuth2, and rate limiting) each represent distinct positions along these dimensions, enabling systematic compar-

ison through categorical analysis using ANOVA. A fourth pattern, centralised API gateway deployment (e.g., Kong), occupies a distinct position on the security function deployment dimension but was excluded from the empirical scope due to methodological constraints; it is addressed as future research in Chapter 5.

2.1.2 Performance Overhead

Performance overhead is defined, following Jain [24], as the additional resource consumption introduced by a system component beyond the theoretical minimum required for core functionality. In the context of API security, overhead represents the additional latency, throughput reduction, and resource utilisation introduced by security mechanisms beyond what an unsecured baseline requires. This definition distinguishes overhead from performance degradation, which implies a broken or abnormal state; overhead is an expected and measurable cost of implementing security functionality.

Three measurement dimensions capture performance overhead comprehensively. Response time (latency) measures the elapsed time between a client sending a request and receiving a complete response, expressed in milliseconds. Throughput measures the number of requests the system can successfully process per unit time, expressed in requests per second. Resource utilisation measures the computational resources consumed during request processing, including CPU usage and memory consumption, expressed as percentages of available capacity. Smith and Williams [25] establish that all three dimensions are necessary for a complete performance assessment, as optimising one dimension may adversely affect others.

2.1.3 High-Tech SME Context

This research defines SMEs using the European Commission criteria [15]: organisations with fewer than 250 employees and annual turnover not exceeding 50 million euros. The qualifier *high-tech* denotes SMEs that develop or are heavily dependent upon software and digital products, where API infrastructure is central to their operations and revenue generation. This qualification is significant because high-tech SMEs possess greater technical capability than general SMEs but remain fundamentally resource-constrained relative to large enterprises.

The SME context serves as the **practical motivating rationale** for this research rather than as a theoretical contribution in itself. Welsh and White [26] established that a small business is not simply a scaled-down version of a large enterprise, SMEs face qualitatively different constraints rather than merely quantitative reductions in resources. This principle motivates why enterprise performance benchmarks cannot be straightforwardly extrapolated to SME contexts and why SME-specific empirical evaluation is necessary. The factual situation of high-tech SMEs, limited computational capacity (often single or few server in-

stances), small or absent dedicated security teams (typically 1 to 5 technical personnel handling multiple responsibilities), and competing priorities between feature delivery and security hardening [27], creates the operational environment in which this research is applicable. SME context therefore scopes and motivates the research without itself constituting the theoretical or empirical gap addressed.

2.1.4 REST Architectural Style

Representational State Transfer (REST) is an *architectural style* for distributed hypermedia systems defined by Fielding [22] in his doctoral dissertation, described as a style rather than a theory, as Fielding himself characterises it, though it carries theoretical underpinnings drawn from network-based software architecture research. REST specifies six architectural constraints: client-server separation, statelessness, cacheability, layered system, code-on-demand (optional), and uniform interface. Two constraints are particularly relevant to the performance implications of API security.

The *statelessness* constraint requires that each request from client to server contains all information necessary for the server to understand and process the request. No session state is stored on the server between requests. This constraint directly shapes authentication design: stateless approaches such as JWT embed all necessary authentication information within the token itself, enabling the server to verify identity without external dependencies. In contrast, approaches requiring server-side state lookup or external service consultation, such as OAuth2 token introspection, introduce architectural departures from strict statelessness.

The *layered system* constraint permits the architecture to be composed of hierarchical layers, with each component unable to see beyond its immediate layer. This enables security to be implemented as an intermediary layer, as in API gateway architectures, where a dedicated component enforces security policies before requests reach backend services. The layered system constraint introduces a fundamental architectural trade-off: intermediary layers enable consolidated policy enforcement and separation of concerns but introduce additional network traversal and processing latency.

2.2 API Security Landscape

The proliferation of API-driven architectures has fundamentally transformed how software systems are designed, deployed, and interconnected. Microservices architectures [1, 28], cloud-native applications, and third-party integrations rely extensively on APIs as the primary mechanism for inter-service and external communication. This architectural shift has simultaneously expanded the attack surface available to adversaries, making API security a critical concern for organisations of all sizes. Pahl and Jamshidi [29] identify authentication

as a cross-cutting concern in microservices deployments, noting that centralised versus distributed authentication strategies create different performance profiles. Berardi et al. [30], in the largest systematic literature review on microservice security to date (290 publications), confirm that while the microservice paradigm has attracted substantial research interest, the literature remains fragmented across venues and predominantly addresses specific scenarios rather than providing comprehensive, empirically grounded security guidance. This fragmentation has a direct consequence for practitioners: when selecting among competing security architectural patterns, there is no unified body of comparative evidence to inform the decision. Instead, practitioners must extrapolate from studies conducted under differing conditions, with differing metrics, against differing baselines, a methodological incompatibility that the present research addresses through controlled comparison under consistent conditions.

2.2.1 API Threat Environment

The OWASP API Security Top 10 [6] provides an authoritative taxonomy of critical API vulnerabilities, identifying broken authentication, broken object-level authorisation, and unrestricted resource consumption among the most prevalent risks. Sun et al. [31] examine key issues in API security research, noting that the rapid growth of API ecosystems has outpaced the development of adequate security measures, particularly for smaller organisations lacking dedicated security expertise. Industry reports corroborate these findings; API-targeted attacks have increased substantially in both frequency and sophistication, with authentication-related vulnerabilities representing a significant proportion of successful breaches [32, 33].

The threat landscape is particularly consequential for SMEs because the financial and reputational impact of a successful API breach can be disproportionately damaging relative to organisational size. Eling and Wirfs [3] document the actual costs of cyber risk events, demonstrating that smaller organisations face higher relative costs per incident. This asymmetric risk profile creates urgency for effective API security but, paradoxically, the resource constraints that make SMEs vulnerable also limit their capacity to implement comprehensive security measures.

2.2.2 API Security Solution Categories

API security solutions can be categorised along the three architectural dimensions introduced in Section 2.1.1. Authentication and authorisation mechanisms, varying in mechanism complexity, form the foundational layer. Stateless token-based authentication using JWT (RFC 7519) [11] enables resource servers to verify client identity through cryptographic signature validation without external dependencies. The token contains encoded claims including issuer, subject, expiration, and custom attributes, signed using symmetric

(HMAC) or asymmetric (RSA, ECDSA) algorithms. Delegated authorisation using OAuth2 (RFC 6749) [13] separates the authorisation decision from the resource server, introducing an authorisation server that issues, manages, and validates access tokens through defined grant flows.

Gateway-based solutions, representing centralised security function deployment, consolidate multiple security functions including authentication, rate limiting, logging, and request transformation within a single architectural component. This pattern centralises security management but introduces an additional network hop and processing stage for every request. The centralised gateway pattern was not included in the empirical scope of this study; it is identified as a priority for future research in Chapter 5.

Traffic control mechanisms, including rate limiting and throttling, address availability protection by restricting request volumes. These mechanisms operate on different information than authentication (request metadata versus identity claims) and serve a distinct security function (availability versus confidentiality and integrity). Rate limiting implementations range from simple fixed-window counters to sophisticated sliding-window algorithms with distributed state management.

Research on these security technologies has primarily focused on detection effectiveness and security properties rather than performance implications. Yarygina and Bagge [8] represent a notable exception, reporting approximately 11% performance overhead from security mechanisms in microservice architectures and identifying authentication and authorisation as the primary sources of latency in service-to-service communication. However, their study did not isolate individual security patterns under controlled resource constraints. While the security capabilities of these solutions are well documented, systematic performance evaluation under realistic conditions, particularly resource-constrained SME environments, remains limited in the literature.

2.3 Performance Engineering Foundations

2.3.1 Software Performance Engineering Principles

Software Performance Engineering (SPE), as formalised by Smith and Williams [25], establishes that system performance is fundamentally determined by architectural decisions rather than solely by implementation quality or hardware specifications. This principle is central to the present research: examining how different security architectural patterns affect performance requires understanding that architectural choices create structural performance characteristics that persist regardless of implementation optimisation.

Jain [24] provides the methodological framework for rigorous performance analysis, establishing principles for experimental design, workload characterisation, and statistical analysis of performance data. Jiang and Hassan [34] extend this foundation with a compre-

hensive survey of load testing methodologies for large-scale software systems, identifying key practices for workload modelling, test environment configuration, and result analysis that inform the experimental approach adopted in this research. Three principles from this methodological foundation are directly applied in the present study. First, controlled experimentation with isolated variables: each security architectural pattern is tested independently against the same baseline, with Docker resource constraints held constant, to enable valid attribution of performance differences to specific architectural choices. Second, multi-metric assessment: latency, throughput, and resource utilisation are captured simultaneously, following Smith and Williams' [25] principle that optimising one dimension may adversely affect others. Third, realistic workload characterisation: load levels are selected to represent operational scenarios ranging from light usage to high-concurrency production loads, informed by Jiang and Hassan's [34] guidance on workload representativeness rather than synthetic extremes.

2.3.2 Security-Performance Trade-off Literature

The fundamental principle that security mechanisms consume computational resources, creating measurable performance overhead, is well established in the literature. Wolter and Reinecke [19] provide a formal treatment of the security-performance trade-off, presenting standard performance metrics alongside proposed security metrics and demonstrating that system parameters can be found that optimise the combination of both, rather than optimising either in isolation. Their work establishes the theoretical basis for quantifying the performance cost of security and shows that this trade-off can be modelled and analysed systematically. Anderson [35] further establishes that security mechanisms impose measurable computational costs that must be treated as first-class architectural concerns rather than implementation afterthoughts.

The trade-off manifests through several mechanisms. Cryptographic operations required for token signing and verification consume CPU cycles proportional to algorithm complexity and key length. Network communication for external validation introduces latency dependent on network conditions and authorisation server load. Request inspection and policy evaluation add processing time to every transaction. These mechanisms are not merely implementation artefacts but architectural consequences of the security design choices described in Section 2.1.1.

A critical gap in the existing trade-off literature is the assumption of adequate computational resources. Most formal analyses and empirical studies evaluate security-performance trade-offs under conditions where resources are abundant relative to demand. Under resource-constrained conditions typical of SME environments, the dynamics of this trade-off may differ qualitatively. When system resources approach saturation, the binding constraint may shift between CPU-intensive and I/O-intensive operations depending on

load conditions, potentially producing non-linear overhead patterns not predicted by analyses assuming resource abundance.

2.3.3 Queueing Theory Foundations

The mathematical relationship between system utilisation and response time provides formal grounding for understanding performance behaviour under load. Little's Law [36] establishes that the average number of items in a queuing system equals the arrival rate multiplied by the average time each item spends in the system ($L = \lambda W$). Applied to API request processing, this relationship explains why response times increase non-linearly as server utilisation approaches capacity: each additional request not only requires its own processing time but also waits longer in the queue as the server handles preceding requests.

This queueing perspective is particularly relevant under SME resource constraints. A system with limited computational capacity (such as the 1 vCPU, 1 GB RAM containers used in this research to simulate SME infrastructure) operates at higher utilisation ratios than enterprise systems handling equivalent workloads. The use of Docker containers as the experimental platform is supported by Felter et al. [18], whose seminal comparison of virtual machines and Linux containers demonstrated that containers achieve near-native performance with less than 5% overhead for CPU and memory-intensive workloads, ensuring that measured performance differences can be attributed to the security mechanisms under test rather than to virtualisation artefacts. As utilisation increases, the system moves into a regime where small differences in per-request processing time, such as those introduced by different security mechanisms, produce amplified differences in observed response time. This mathematical property explains why security overhead that appears negligible under low utilisation may become significant under realistic load conditions.

2.4 SME-Specific Context and Constraints

This section provides the operational background that motivates the research scope. SME context is not itself a theoretical or empirical gap in this work; rather, it establishes the practical environment in which the research is applicable and explains why enterprise-derived evidence is insufficient for SME practitioners. The theoretical mechanisms through which resource constraints produce qualitatively different performance dynamics are addressed in Section 2.6.3.

2.4.1 Resource Constraints and Their Implications

Research examining cybersecurity in SME contexts consistently identifies resource constraints as fundamental challenges [14, 27]. Chidukwani et al. [5] confirm that despite

representing over 90% of businesses globally, SMEs remain underserved by cybersecurity research, with most frameworks and tools designed for larger organisations. Carias et al. [37] further demonstrate the practical difficulty of operationalising security in SME environments, finding that even well-intentioned security frameworks require significant adaptation to become viable under real SME resource conditions. The European Union Agency for Cybersecurity [4] corroborates these findings at a European scale, reporting that 90% of surveyed SMEs stated cybersecurity issues would have serious negative impacts on their business within a week, yet many lack dedicated budgets, specialist personnel, and suitable guidelines. SMEs face limited IT budgets that prevent implementation of enterprise-grade infrastructure, small technical teams handling multiple responsibilities including security alongside development and operations, and infrastructure operating on minimal cloud instances. These constraints create heightened sensitivity to performance overhead: security solutions consuming significant CPU or memory may be infeasible given limited capacity, latency increases directly impact user experience and competitive positioning, and the cost of scaling infrastructure to compensate for security overhead may be prohibitive.

Barlette and Fomin [14] examined the suitability of information security management standards for SMEs, finding that most frameworks assume organisational capabilities and resources unavailable to typical SMEs. This misalignment creates a gap where technically effective security solutions remain practically infeasible for SME adoption. The implication for the present research is that performance overhead data from enterprise benchmarks cannot be assumed applicable to SME contexts; the resource constraint environment may qualitatively alter how security architectural patterns perform. This practical inapplicability of enterprise evidence is the primary motivating argument for conducting SME-specific empirical research.

2.4.2 SME Security Decision-Making Behaviour

A recurring pattern in the practitioner literature is that SMEs rarely conduct systematic evaluation of security solution performance before adoption. Organisations tend to adopt authentication and API security configurations based on familiarity, framework defaults, or ease of implementation rather than evidence-based comparison [14, 27]. Availability concerns frequently take precedence over security hardening in SME decision-making, driven by service level agreement requirements and delivery pressures. This pattern is consistent with satisficing behaviour: organisations establish aspiration levels for acceptable performance without the empirical data necessary for optimisation. The theoretical interpretation of this decision-making pattern is addressed in Section 2.6, where it is examined through the lens of bounded rationality and satisficing theory.

2.5 State of the Art: API Security Performance

The existing literature on API security performance is characterised by several methodological approaches, each with significant limitations for the present research context. Studies examining authentication mechanism performance typically evaluate a single mechanism in isolation, measuring the overhead introduced by JWT or OAuth2 implementations without providing controlled comparisons under identical conditions. This makes cross-study comparison unreliable due to differences in hardware configurations, software stacks, workload characteristics, and measurement methodologies.

Studies that do compare multiple authentication mechanisms generally employ enterprise-grade infrastructure with abundant computational resources, multiple CPU cores, and generous memory allocations. While these studies provide valuable baseline data for enterprise architects, their findings cannot be extrapolated to SME contexts where resource constraints may qualitatively alter performance characteristics. A security mechanism that introduces negligible overhead with eight CPU cores and sixteen gigabytes of memory may exhibit fundamentally different behaviour when constrained to a single CPU core and one gigabyte of memory. Chatterjee and Prinz [7], for example, evaluate OAuth2 with KeyCloak in a microservice architecture and provide useful empirical data on authentication overhead, but their experimental environment does not impose the resource constraints characteristic of SME deployments, limiting the applicability of their findings to the present research context.

API gateway performance studies similarly focus on enterprise deployment scenarios, evaluating throughput capacity and latency under conditions that assume horizontal scaling capability and dedicated infrastructure. De Almeida and Canedo [10] provide the most current systematic review of authentication and authorisation mechanisms in microservices architectures, confirming that JWT and OAuth2 are the two dominant implementation patterns but noting that empirical performance data under constrained deployment conditions remains a gap in the literature. Rate limiting performance literature is largely embedded within broader API management discussions rather than isolated as an independent performance variable. Across the surveyed literature, no study was identified that systematically compares multiple API security architectural patterns under controlled, resource-constrained conditions representative of SME environments.

The preceding review has established the current state of knowledge across API security, performance engineering, and SME-specific constraints, identifying both well-established foundations and significant gaps. The following section presents the theoretical framework that integrates these foundations into a coherent structure guiding the research design, connecting established theories to the specific predictions tested in this study.

2.6 Theoretical Framework

This section presents the theoretical framework guiding the research, classified using Gregor's [38] taxonomy of theory types in information systems. Gregor identifies five theory types: Type I (Analysis) theories describe and classify phenomena; Type II (Explanation) theories explain how and why phenomena occur; Type III (Prediction) theories forecast outcomes; Type IV (Explanation and Prediction) theories combine explanatory and predictive power; and Type V (Design and Action) theories prescribe methods for achieving desired outcomes. This research draws on four primary theories and two complementary theories, each serving a distinct role within the research design.

Table 2.2: Theoretical Framework Summary

Theory	Gregor Type	Role in This Research
REST Architectural Style	Type I + V	Defines architectural design space; identifies security dimensions
Security-Performance Trade-off Theory	Type III–IV	Predicts and explains overhead patterns across architectural patterns
Resource-Based SME Constraint Theory	Type IV	Explains why SME context produces qualitatively different results
Software Performance Engineering	Type V	Prescribes measurement methodology and experimental rigour
Satisficing / Bounded Rationality	Type II	Explains practitioner decision-making behaviour
Theory of Constraints	Type IV	Explains bottleneck migration under resource constraints

2.6.1 REST Architectural Style (Type I + Type V)

Fielding's [22] REST *architectural style* serves a dual role in this research. As a Type I (Analysis) framework, it provides a descriptive and classificatory structure for characterising the architectural constraints that govern API design. The statelessness, layered system, and client-server constraints establish the design space within which security solutions must operate, defining the architectural properties that distinguish one security approach from another. As a Type V (Design and Action) framework, REST provides prescriptive guidance for how distributed systems should be architected, with specific implications for where and how security functions should be implemented.

The application of REST to this research is direct: the three independent variable dimensions (authentication mechanism complexity, security function deployment pattern, and traffic control mechanism) are derived from REST's architectural constraints. Statelessness

informs the distinction between self-contained and delegated authentication. Layered systems inform the distinction between distributed and centralised security deployment. The uniform interface constraint, requiring standardised interaction patterns, informs why traffic control operates as a distinct architectural concern. REST thus provides the theoretical basis for the independent variable conceptualisation, predicting that architecturally distinct security implementations will exhibit measurably different performance characteristics.

2.6.2 Security-Performance Trade-off Theory (Type III–IV)

Security-performance trade-off theory, as formalised by Wolter and Reinecke [19], establishes that security mechanisms introduce measurable computational costs and that optimal system configuration involves balancing security protection against performance requirements. This theory serves both explanatory and predictive functions (Type III–IV) in this research.

The explanatory function addresses why different security architectural patterns produce different overhead profiles. Stateless authentication (JWT) introduces primarily CPU-bound overhead through cryptographic signature verification, where processing cost scales with algorithm complexity and token payload size. Delegated authorisation (OAuth2) introduces primarily I/O-bound overhead through network communication with the authorisation server, where cost depends on network latency and authorisation server responsiveness. Each pattern produces a distinct overhead profile because each engages different computational resources in different proportions.

The predictive function generates testable hypotheses for this research. Under resource-constrained conditions where CPU is the binding constraint, CPU-bound mechanisms should saturate available resources faster than I/O-bound mechanisms, potentially producing crossover patterns where the theoretically simpler mechanism exhibits higher overhead at elevated loads. This prediction, derived from the interaction of security-performance trade-off theory with resource constraint theory (Section 2.6.3), is tested rigorously in the full experimental programme.

2.6.3 Resource-Based SME Constraint Theory (Type IV)

The resource-based perspective on SME constraints draws on Welsh and White's [26] foundational argument that small businesses face qualitatively different environments than large enterprises, and Barney's [39] resource-based view establishing that organisational capabilities are constrained by available resources. In this research, this theory serves two roles: it *constrains the experimental context* by explaining why the 1 vCPU, 1 GB RAM conditions are the appropriate test environment, and it *provides the explanatory mechanism* for why performance dynamics under these constraints may differ qualitatively from enterprise benchmarks. The SME context is thus not itself a research gap; it is the theoretically

grounded reason why existing enterprise-focused evidence cannot be straightforwardly applied to SME decision-making.

The explanatory mechanism operates as follows. With abundant computational resources (enterprise context), all security architectural patterns operate well below resource saturation, overhead differences between patterns are relatively small, and architectural choice has limited practical significance for performance. Under constrained resources (SME context), security mechanisms compete for limited CPU, memory, and I/O bandwidth, overhead differences are amplified as the system operates closer to saturation, and architectural choice becomes consequential because the margin between acceptable and unacceptable performance narrows substantially. This theory predicts that the same set of security solutions will exhibit qualitatively different performance characteristics when deployed in SME-constrained environments compared to enterprise environments, making SME-specific empirical evaluation necessary.

2.6.4 Software Performance Engineering (Type V)

Software Performance Engineering (SPE), as established by Smith and Williams [25] and operationalised through Jain's [24] performance analysis methodology, serves as a Type V (Design and Action) theory prescribing how performance evaluation should be conducted. SPE theory establishes several principles applied in this research: performance characteristics should be evaluated through controlled experiments isolating the variable of interest; multiple performance metrics should be assessed simultaneously to capture trade-offs between latency, throughput, and resource utilisation; workload conditions should represent realistic operational scenarios rather than synthetic extremes; and results should be analysed using appropriate statistical methods to establish significance. This theory justifies the experimental design choices presented in Chapter 3, providing methodological rigour grounded in established performance engineering practice.

2.6.5 Complementary Theories

Satisficing and Bounded Rationality (Type II). Simon's [40, 41] theory of bounded rationality and satisficing provides an explanatory framework for understanding SME security decision-making behaviour documented in the practitioner literature [14, 27]. SMEs tend to adopt authentication defaults based on familiarity rather than evidence-based evaluation, consistent with satisficing: decision-makers establish aspiration levels [41] without the empirical data necessary for optimisation. This theory positions the practical contribution of this research: by providing empirical performance data under SME-realistic conditions, the research enables practitioners to move from satisficing toward more informed architectural decision-making.

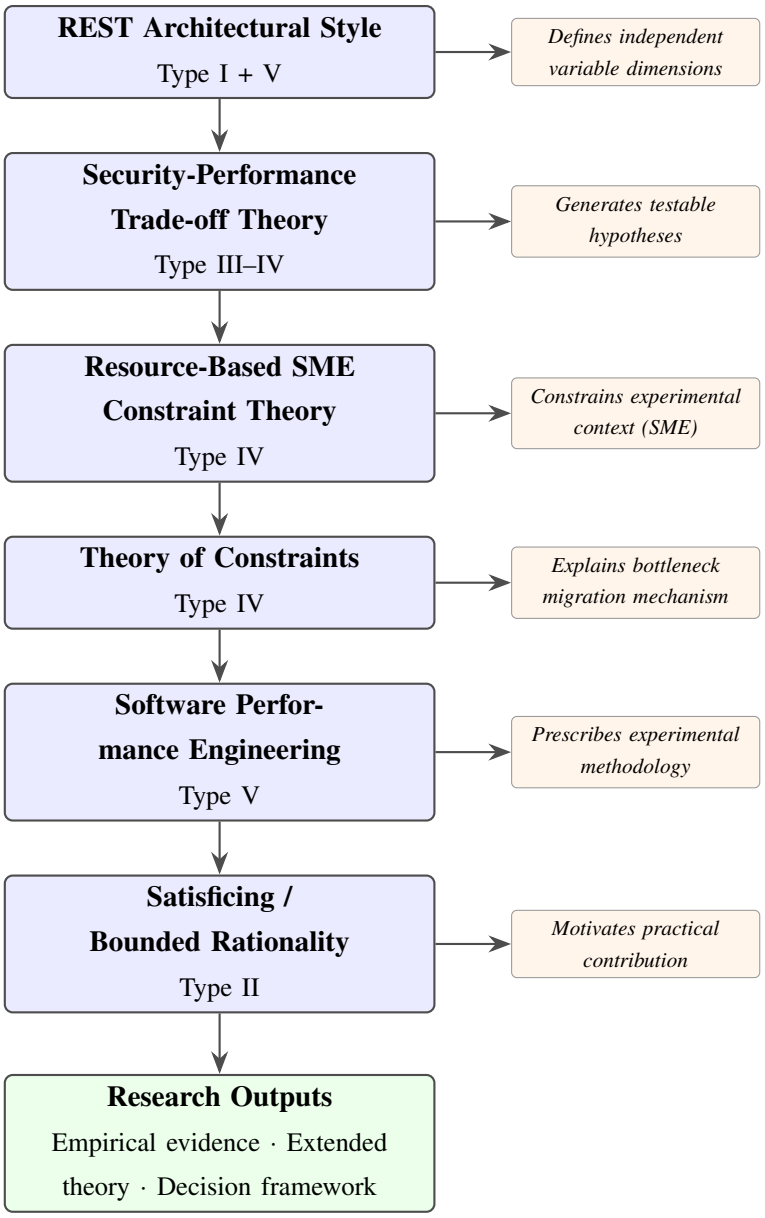
Theory of Constraints (Type IV). Goldratt's [42] Theory of Constraints (TOC) provides an explanatory and predictive framework for interpreting the performance patterns observed in this research. TOC establishes that system throughput is limited by a single binding constraint at any point in time, and that improving non-constraint components yields no system-level benefit. Applied to API security under resource constraints, TOC predicts that the binding constraint can shift between CPU-intensive and I/O-intensive operations as load increases. At low load, CPU demand from JWT cryptographic verification is minimal and I/O latency from OAuth2 external calls dominates, making JWT faster. At high load, JWT's CPU demand scales and may saturate the CPU constraint, while OAuth2's I/O costs are partially amortised through connection reuse, potentially making OAuth2 more efficient. This bottleneck migration mechanism is empirically confirmed by the experimental findings reported in Chapter 4 and discussed in Chapter 5.

2.6.6 Theory Integration and Research Model

The six theories integrate into a coherent research framework. REST architectural style identifies the architectural dimensions that define the independent variable and establishes the design space within which security solutions operate. Security-performance trade-off theory predicts that different positions within this design space will produce measurably different overhead profiles, generating testable hypotheses. Resource-based SME constraint theory explains why these predictions must be tested under SME-realistic conditions and predicts that overhead patterns will differ qualitatively from enterprise benchmarks. The Theory of Constraints explains the mechanism through which these qualitative differences emerge, specifically the migration of system bottlenecks under varying load. SPE theory prescribes the rigorous experimental methodology necessary for valid measurement. Satisficing theory explains why the resulting empirical data constitutes a practical contribution, addressing the current information deficit in SME security decision-making.

Figure 2.1 illustrates how these theories connect within the research design, showing the logical flow from architectural style through hypothesis generation, contextual constraints, and methodology to the practical contribution.

Figure 2.1: Integrated Research Model: Theory-to-Design Logic



Together, these theories predict that: (a) different API security architectural patterns will exhibit statistically significant performance differences under SME-realistic constraints; (b) the relative performance ranking of security patterns will change across load conditions due to bottleneck migration; and (c) the magnitude of overhead differences will be practically significant relative to a conservative 10 ms acceptability threshold (set an order of magnitude below the established 100 ms HCI perceptual benchmark). These predictions are tested through the experimental programme presented in Chapter 3.

2.7 Research Gap and Expected Contribution

The literature review reveals three intersecting gaps that this research addresses. First, an **empirical gap**: while API security solutions are well characterised in terms of security properties, no identified study systematically compares the performance of multiple API security architectural patterns under controlled conditions representative of SME resource constraints. Existing performance studies either evaluate single mechanisms, use enterprise-scale infrastructure, or lack the controlled experimental design necessary for valid comparison.

Second, a **theoretical gap**: security-performance trade-off theory predicts that different architectural patterns will produce different overhead profiles, but these predictions have not been empirically validated under resource-constrained conditions. The full experimental programme reported in Chapter 4 tests this prediction and, as discussed in Chapter 5, extends security-performance trade-off theory by demonstrating context-dependent performance characteristics not predicted by analyses assuming resource abundance.

Third, a **practical gap**: SMEs lack evidence-based guidance for API security architectural decisions. This practical gap serves as the primary motivating argument for the research: the factual operational situation of SMEs, constrained infrastructure, small technical teams, and disproportionate exposure to cyber risk costs, makes the absence of SME-specific performance evidence a genuine and pressing operational problem. Practitioners appear to satisfice rather than optimise their security choices because performance data specific to their operational context does not exist. Current guidance, derived from enterprise benchmarks or vendor documentation, may be misleading when applied to SME infrastructure. By providing empirical overhead measurements under SME-realistic conditions, this research enables more informed architectural decision-making.

The expected contributions span three domains. Empirically, this research provides the first controlled comparison of API security solution performance under SME-realistic resource constraints across multiple load conditions. Theoretically, it validates and extends security-performance trade-off theory in resource-constrained contexts and offers an empirically grounded application of the Theory of Constraints to API security architecture. Practically, it delivers an evidence-based decision framework enabling SMEs to select security architectural patterns informed by quantitative performance data relevant to their operational environment.

Chapter 3

Research Methodology

This chapter describes the research design, experimental infrastructure, variable operationalisation, data collection procedures, and validity considerations. The methodology is primarily quantitative, employing a controlled laboratory experiment as the principal research instrument, with a supplementary qualitative strand planned for external validation.

3.1 Research Design

This study adopts a positivist epistemological stance, seeking causal relationships between authentication architectural characteristics and measurable API performance outcomes through controlled experimentation, consistent with established practices in empirical software engineering [16, 43]. The primary research instrument is a controlled laboratory experiment.

The methodology addresses three research questions:

RQ1. What is the measurable performance overhead introduced by different API authentication mechanisms under SME-realistic resource constraints?

RQ2. How does the interaction between authentication architectural pattern and concurrent user load affect API response time and throughput?

RQ3. Under what conditions does authentication overhead remain within a practically acceptable threshold for SME-hosted API deployments?

The experimental component follows a 4×5 factorial design, crossing four configurations (three security patterns plus an unsecured baseline) against five concurrent user load levels with five replications per cell, totalling 100 runs.

3.2 Conceptualisation of Variables

3.2.1 Independent Variable: Authentication Architectural Pattern

The independent variable is *authentication architectural pattern*, operationalised as a nominal categorical variable with four levels (no-auth baseline, JWT, OAuth2, and Redis-backed

rate limiting). The baseline constitutes the fourth factor level, enabling within-ANOVA partitioning of authentication variance. Three architectural dimensions differentiate the authenticated patterns:

1. **State model**: stateless (credentials embedded in token) versus stateful (server-side state required for verification).
2. **Verification locus**: in-process (within the application) or out-of-process I/O-bound (network call to external service).
3. **Computational bottleneck type**: CPU-bound (cryptographic computation) or I/O-bound (network round-trip or in-memory lookup).

Table 3.1: Independent Variable Operationalisation: Authentication Patterns by Architectural Characteristics

Pattern	Implementation	State model	Verification locus	Bottleneck type
No-auth baseline	None	Stateless	None (control)	None
JWT	HMAC-SHA256	Stateless	In-process (CPU)	CPU-bound
OAuth2	Token introspection	Stateful	Out-of-process (I/O)	I/O-bound
Rate-limiting	Redis sliding window	Stateful	Out-of-process (I/O)	I/O-bound

Note. The centralised/gateway deployment pattern constitutes a third position on the security function deployment dimension but falls outside the empirical scope of this study. All three authenticated patterns tested occupy the distributed deployment position on that dimension.

3.2.2 Dependent Variables

Three dependent variables are measured per condition:

1. **Mean response time** ($\bar{x}$, ms): arithmetic mean of `api_data` endpoint response times across all requests in a run.
2. **Authentication overhead** (Δms): mean response time minus baseline mean at the same concurrent user load level. Primary dependent variable for between-pattern comparisons.
3. **Observed throughput** (req/s): successful `api_data` requests per second, averaged across a run.

3.2.3 Control Variables

Held constant across all conditions: container resource limits (1 vCPU, 1 GB RAM); Node.js 18.x LTS and Express 4.x; identical `doWork()` business logic; test duration (120 s); ramp-up rate (users/10 users/s); warm-up period (60 s); cool-down between replications (30 s); Locust generator running on the host OS outside containerised resource limits.

3.3 Experimental Infrastructure

3.3.1 Hardware and Network Configuration

The experimental environment runs on a single physical server (Ubuntu Server 24.04 LTS with Docker Engine), hosting all containerised API services with enforced resource limits (`cpus: "1.0"`, `memory: 1g` per container), replicating the constrained infrastructure profile characteristic of SME-hosted deployments. The **load generator** (Locust 2.x) runs directly on the host operating system outside the containerised environment. Because Locust and the containers share the same physical CPU and memory, resource contention at higher load levels is a potential confound; this is acknowledged as an internal validity limitation in Section 3.7. However, the container-level resource caps ensure that each API service is individually constrained regardless of host-level activity, and the single-machine topology is itself representative of resource-constrained SME deployments where load testing and production services commonly coexist.

3.3.2 Security Pattern Implementations

Four containerised implementations expose identical endpoints (`GET /api/data` and `GET /health`) with identical business logic, differing only in their authentication layer:

Baseline (no-auth): Node.js/Express API with no authentication. Performance reference for overhead computation. Port 8080.

JWT: Stateless HMAC-SHA256 token authentication using `jsonwebtoken`. Tokens verified on every request; no server-side state. Port 8081.

OAuth2: Token introspection using `oauth2-server`. An in-process authorisation server issues opaque tokens; each API request triggers an introspection call against this server. A deliberate 5 ms synthetic delay simulates the latency of external token introspection, representing a controlled experimental parameter rather than a real external authorisation server call. This design choice preserves internal validity while introducing a documented limitation discussed in Section 3.7. Port 8082.

Rate-limiting: Redis-backed sliding window rate limiter using `rate-limiter-flexible`. Redis container stores per-IP request counters with a sufficiently high limit

(1,000,000 requests/minute) so that no requests are blocked during experiments, isolating middleware overhead from rejection effects. Port 8083.

3.3.3 Containerisation and Orchestration

Docker Compose defines and orchestrates the environment. Each experimental session starts only the containers required for the current pattern and its dependencies, stopping and removing them afterwards to prevent state accumulation. Redis state is flushed with `redis-cli FLUSHALL` between rate-limiting sessions.

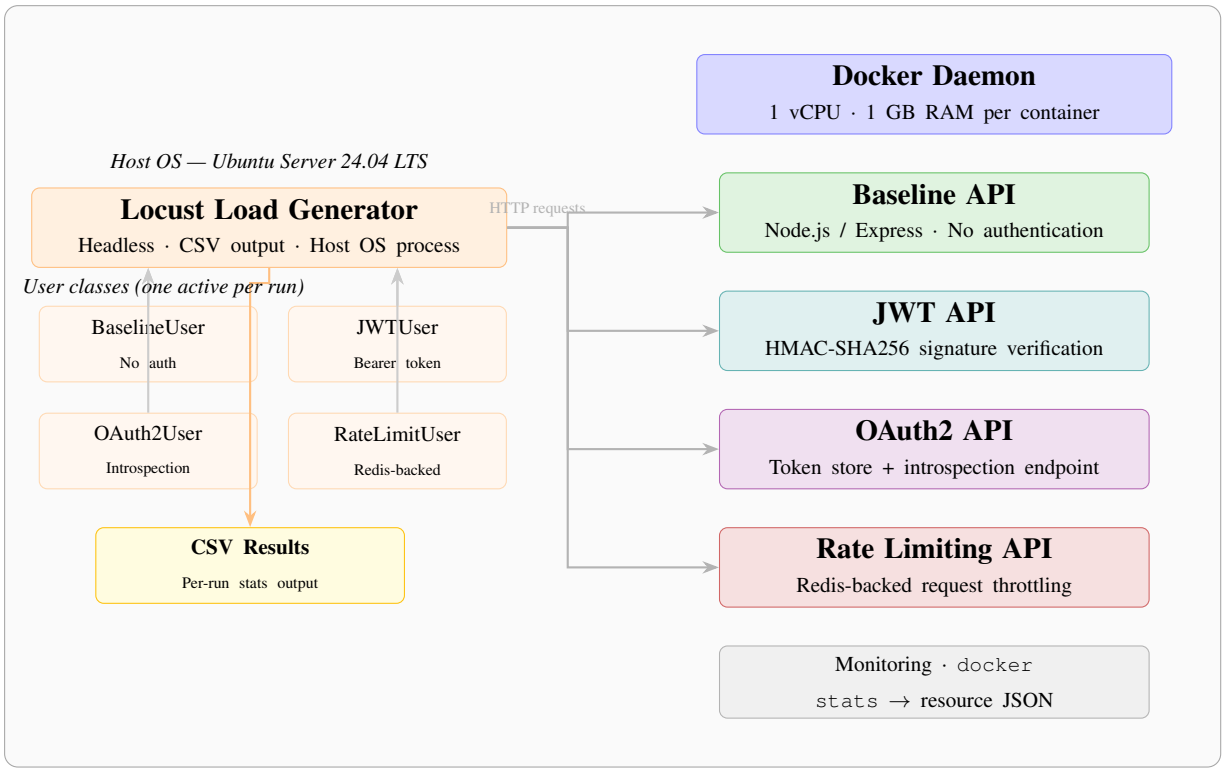


Figure 3.1: Experimental Infrastructure: Single-Server Setup with Host-Level Load Generator and Four Containerised Security Pattern Implementations with Enforced Resource Limits (1 vCPU, 1 GB RAM per Container)

3.4 Experimental Design and Procedure

3.4.1 Factorial Design

The experiment follows a 4×5 between-subjects factorial design:

- **Factor A: Security pattern**: 4 levels (Baseline, JWT, OAuth2, rate-limiting)
- **Factor B: Concurrent user load**: 5 levels (15, 75, 150, 300, 600 users)
- **Replications**: 5 per cell

- **Total runs**: $4 \times 5 \times 5 = 100$ total

Load levels are expressed in *concurrent users* rather than requests per second because concurrent users constitute the experimentally controlled variable; Locust spawns a specified number of virtual users each issuing requests according to defined task weights and wait times. Observed throughput (req/s) is a measured outcome, not a directly controlled input. The levels (15, 75, 150, 300, 600) span minimal to high-concurrency load, each separated by a factor of approximately 2–5×, providing adequate resolution to observe non-linear scaling effects.

3.4.2 Load Test Configuration

Locust is configured with: task weights of 10 (`api_data`) to 1 (`health`), reflecting realistic usage where health checks are infrequent; wait time of `between(0.01, 0.02)` seconds per user; spawn rate of users$/10$ users/second; and 120-second test duration. The minimal inter-request wait time produces sustained, high-frequency load stress appropriate for identifying performance boundaries under SME-constrained conditions. Authentication credentials are obtained once at the start of each load level session and reused across all five replications, reflecting a token-caching deployment pattern. Results are saved as Locust CSV statistics files (`run_N_stats.csv`).

3.4.3 Execution Protocol

A master shell script (`run_experiment.sh`) automates the protocol:

1. Start containers; wait for health verification
2. Execute 60-second warm-up to achieve steady-state JIT compilation
3. For each load level: obtain credentials; for each replication ($r = 1 \ldots 5$): execute 120-second Locust run, save CSV, wait 30-second cool-down
4. Stop and remove containers; sleep 5 seconds before next pattern

The script implements idempotent skip logic: if a results file already exists that run is skipped, allowing safe re-execution after SSH connection interruptions. Experiments were executed within `screen` sessions to maintain persistent shell processes.

3.4.4 Research Hypotheses

The factorial design tests four hypotheses, each linked to the research questions and to specific terms in the ANOVA model (Equation 3.2). Figure 3.2 presents the research model showing all hypotheses simultaneously.

H1 (RQ1 — Main effect of security pattern). Derived from Security-Performance Trade-off Theory (Section 2.6), which predicts that mechanisms differing in computational cost produce different overhead:

> H_{1_0}: Authentication overhead does not differ significantly across security patterns. Formally, $\alpha_i = 0$ for all i.
>
> H_{1_a}: At least one security pattern produces significantly different authentication overhead.

H2 (RQ2 — Interaction effect). Derived from the integration of Security-Performance Trade-off Theory with the Theory of Constraints (Section 2.6), which predicts bottleneck migration from I/O to CPU as load rises and thus a load-dependent reversal in pattern ranking:

> H_{2_0}: The effect of security pattern on authentication overhead is independent of concurrent user load level. Formally, $(\alpha\beta)_{ij} = 0$ for all i, j.
>
> H_{2_a}: The effect of security pattern on authentication overhead depends on the concurrent user load level (i.e., the patterns scale differently under increasing load).

H3 (RQ2 — Main effect of load level). Grounded in queueing theory (Section 2.6), which predicts non-linear response-time growth as utilisation approaches saturation under constrained capacity:

> H_{3_0}: Authentication overhead does not differ significantly across load levels. Formally, $\beta_j = 0$ for all j.
>
> H_{3_a}: At least one load level produces significantly different authentication overhead.

H4 (RQ3 — Practical significance threshold). Operationalising the practical-significance criterion against the literature-anchored 10 ms bound (Section 3.5):

> H_{4_0}: Mean authentication overhead is at most 10 ms for all pattern $\times$ load combinations. Formally, $\mu_{\Delta_{s,l}} \leq 10$ for all s, l.
>
> H_{4_a}: At least one pattern $\times$ load combination produces mean overhead exceeding 10 ms.

H1–H3 are tested via the two-way factorial ANOVA (Equation 3.2); H4 is assessed through descriptive classification of each pattern $\times$ load condition as within-threshold (✓, $\Delta \leq 10$ ms) or exceeding-threshold ($\times$, $\Delta > 10$ ms).

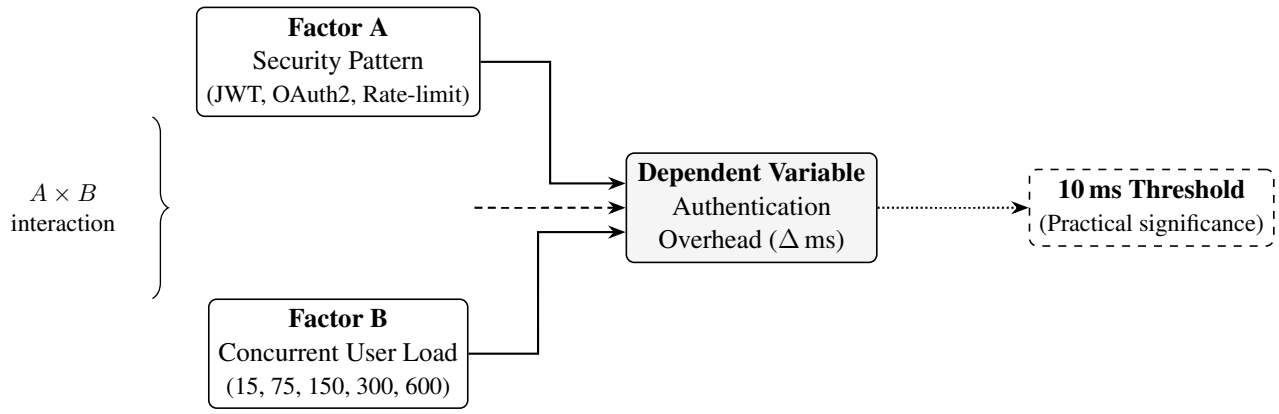


Figure 3.2: Research model showing the four hypotheses tested in the factorial experiment. Solid arrows represent main effects (H1, H3); the dashed arrow represents the interaction effect (H2); the dotted arrow represents the practical significance assessment (H4) against the conservative 10 ms threshold.

Table 3.2: Hypothesis–Analysis Alignment: Data Types, Statistical Tests, and Assumption Checks

Hyp.	IV type	DV type	Data produced	Analysis	Assumptions
H1	Nominal (4 levels)	Continuous (interval)	Δ ms per cell ($n = 100$)	ANOVA main effect α_i	Shapiro-Wilk, Levene's
H2	Nominal $\times$ Ordinal	Continuous (interval)	Δ ms across 15 cells	ANOVA interaction $(\alpha\beta)_{ij}$	Shapiro-Wilk, Levene's
H3	Ordinal (5 levels)	Continuous (interval)	Δ ms per cell ($n = 100$)	ANOVA main effect β_j	Shapiro-Wilk, Levene's
H4	—	Continuous vs. threshold	Δ ms vs. 10 ms criterion	Descriptive classification per cell	—

Note. All hypotheses share the same dependent variable (authentication overhead, Δ ms) computed per Equation 3.1. H1–H3 are tested simultaneously within the two-way factorial ANOVA (Equation 3.2). H4 is assessed independently per cell. Where ANOVA assumptions are violated, Kruskal-Wallis sensitivity checks confirm directional consistency (Section 3.5). Effect sizes are reported as partial η_p^2 for H1–H3.

3.5 Quantitative Data Analysis

3.5.1 Overhead Computation

Authentication overhead per condition is computed as:

$$\Delta_{s,l} = \bar{x}_{s,l} - \bar{x}_{\text{baseline},l} \tag{3.1}$$

where $\bar{x}_{s,l}$ is the mean response time for security pattern s at load level l, and $\bar{x}_{\text{baseline},l}$ is the baseline mean at the same load level. This normalisation isolates authentication cost from load-induced latency affecting all patterns equally.

3.5.2 Statistical Analysis

A two-way factorial ANOVA tests for main effects of security pattern and load level, and their interaction:

$$\Delta_{ijr} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \varepsilon_{ijr} \tag{3.2}$$

Effect sizes are reported as partial eta-squared (η_p^2). Prior to ANOVA, Shapiro-Wilk tests normality per cell and Levene's test checks homogeneity of variance. Where assumptions are violated, the robustness of balanced ANOVA to moderate violations is considered [44]. Sensitivity checks using Kruskal-Wallis confirm directional consistency. Post-hoc pairwise comparisons use Tukey's HSD, controlling family-wise error at $\alpha = .05$. Analysis was implemented using `pingouin` (v0.5.5) and `statsmodels`.

3.5.3 SME Acceptability Threshold

A 10 ms overhead threshold is applied as a practical significance criterion. The foundational HCI literature establishes 100 ms as the threshold below which system response is perceived as instantaneous in interactive systems [45, 46], a finding reaffirmed by a review spanning four decades of system response-time research [47]. The 10 ms criterion adopted here is therefore a deliberately conservative bound, an order of magnitude below the established 100 ms perceptual threshold—ensuring that any authentication overhead classified as within-threshold remains comfortably imperceptible even when added to other components of the response path. This provides a stringent, literature-anchored benchmark against which measured overhead can be assessed for practical relevance. Each condition is classified as within-threshold ($\Delta \leq 10$ ms, ✓) or exceeding-threshold ($\Delta > 10$ ms, ×).

3.6 Ethical Considerations

This research involves no human subjects. All data were generated through automated load testing against containerised API implementations. No personal data were collected or processed at any stage of the experimental programme.

3.7 Validity Considerations

3.7.1 Internal Validity

Supported by: identical container resource limits, runtime versions, and business logic across all patterns; per-session container lifecycle preventing state accumulation; five replications enabling variance estimation and outlier detection; and a 60-second warm-up achieving steady-state JIT compilation before measurement. Because the load generator (Locust) runs on the same physical host as the containerised services, CPU and memory contention between Locust and the system under test is a potential confound, particularly at higher concurrency levels. This is partially mitigated by Docker's per-container resource caps, which enforce the 1 vCPU / 1 GB ceiling regardless of host-level load, and by the consistent application of this topology across all conditions so that any contention effect is uniform rather than pattern-specific. The ascending load order within each solution session may produce warm-cache effects at higher levels; this is partially mitigated by the 30-second cool-down periods between replications.

3.7.2 External Validity

Constrained by: the specific 1 vCPU / 1 GB RAM profile, which represents a common but not universal SME hosting configuration (findings may not generalise to multi-core environments where CPU-bound JWT verification could be parallelised); LAN-only network topology producing lower and more stable latency than internet-facing production traffic; the synthetic OAuth2 introspection delay (5 ms), which is a controlled experimental parameter rather than a real external authorisation server; token lifecycle simplification (credentials obtained once per session do not capture token refresh overhead); single-node deployment with no horizontal scaling; and the absence of practitioner interview validation, which is identified as a future external-validation strand and does not affect the validity of the experimental findings reported here.

3.7.3 Construct Validity

Supported by the architectural conceptualisation of the independent variable, grounding pattern categorisation in theoretically meaningful dimensions (Section 3.2.1) rather than

technology labels. The concurrent user count constitutes the controlled load variable; observed throughput is a measured outcome, meaning the five load levels represent different absolute throughput conditions for different patterns. This is acknowledged in the results chapter where both concurrent user count and observed throughput are reported.

3.7.4 Reliability

Supported by the automated scripted protocol (`run_experiment.sh`), idempotent run management, and the five-replication design. The complete setup, including Docker Compose configuration, Locust scripts, and analysis pipeline is documented in Appendix A and archived for independent replication. Full statistical output is provided in Appendix B. Low standard deviations across replications provide empirical evidence of measurement consistency. To assess the portability of the measurement framework, the full 100-run experimental protocol was executed without modification on a second physical machine (Intel Core i7-10610U laptop, Parrot OS, identical Docker resource constraints). The framework transferred without modification and produced the same qualitative pattern—JWT overhead diverging sharply above 75 concurrent users while OAuth 2.0 and rate-limiting remained within statistical indistinguishability—demonstrating that the experimental protocol is portable across x86-64 hardware configurations and providing a concrete starting point for the independent verification of the LDAOC that we invite.

3.8 Summary

This chapter described a positivist quantitative research design built around a 4×5 factorial controlled experiment. The independent variable was operationalised along three architectural dimensions (state model, verification locus, and computational bottleneck type), with four factor levels (baseline, JWT, OAuth2, and rate-limiting). The experimental infrastructure comprised four containerised API implementations under SME-realistic resource constraints (1 vCPU, 1 GB RAM), tested across five concurrent user load levels with five replications per condition (100 total runs). The quantitative experiment is the primary and self-sufficient component of the research design, capable of answering all three research questions independently. Chapter 4 presents the experimental findings.

Chapter 4

Results and Analysis

This chapter presents the empirical findings from the controlled experimental evaluation of three security pattern implementations against an unsecured baseline under SME-realistic resource constraints. Results are structured around the four hypotheses (H1–H4) introduced in Section 3.4.4, which map directly onto the three research questions. Section 4.1 presents descriptive statistics and the interaction plot that anchors the chapter. Sections 4.2–4.5 report the inferential tests. Section 4.6 formalises the crossover mechanism, and Section 4.7 synthesises the four principal findings.

All experiments were conducted on a single physical server (Intel Core i5-8500, 16 GB RAM, Ubuntu Server 24.04 LTS, Docker Engine v28.5.1), with each containerised API service allocated 1 vCPU and 1 GB RAM. Locust 2.x ran on the host OS outside the containerised environment. Raw per-run measurements are tabulated in Appendix C.

4.1 Descriptive Statistics and Interaction Overview

Readers seeking the central finding may proceed directly to Figure 4.1 and Section 4.6; the intervening sections provide the statistical foundation for that interpretation.

The full experimental matrix comprised 100 Locust load test runs across four configurations (no-auth baseline, JWT, OAuth2, and Redis-backed rate-limiting), five concurrent user load levels (15, 75, 150, 300, and 600 users), and five replications per condition. Table 4.1 presents the complete dataset; Figure 4.1 presents the interaction plot that serves as the primary visual anchor for this chapter.

The interaction plot (Figure 4.1) reveals the headline finding immediately: JWT overhead diverges sharply from both OAuth2 and rate-limiting between 15 and 75 concurrent users, inverting the rank ordering observed at minimal load. This visual pattern is the primary motivation for H2 (interaction effect) and is statistically confirmed in Section 4.2.

Table 4.1: Complete Experimental Results: Mean Response Time, Authentication Overhead, and Throughput by Security Pattern and Concurrent User Load ($n = 5$ replications per cell)

Solution	**Users**	**Mean (ms)**	σ	Δ **(ms)**	Δ **(%)**	**Tput (r/s)**	n	$\leq$**10 ms**
Baseline	15	3.12	0.36	—	—	702.5	5	—
	75	23.76	0.10	—	—	1471.4	5	—
	150	66.92	0.95	—	—	1419.0	5	—
	300	162.66	1.23	—	—	1385.0	5	—
	600	360.01	2.35	—	—	1351.4	5	—
JWT	15	3.23	0.03	0.11	3.4%	710.8	5	✓
	75	44.49	0.40	20.73	87.2%	1095.2	5	×
	150	104.31	0.33	37.39	55.9%	1096.2	5	×
	300	222.32	3.92	59.67	36.7%	1089.9	5	×
	600	476.85	6.89	116.84	32.5%	1062.6	5	×
OAuth2	15	8.22	0.33	5.10	163.4%	563.7	5	✓
	75	27.52	0.24	3.76	15.8%	1407.5	5	✓
	150	71.02	0.76	4.10	6.1%	1385.5	5	✓
	300	169.09	0.80	6.43	4.0%	1352.9	5	✓
	600	377.36	2.00	17.35	4.8%	1301.4	5	×
Rate-Limiting	15	3.64	0.28	0.52	16.7%	687.6	5	✓
	75	27.27	0.21	3.51	14.8%	1391.0	5	✓
	150	71.57	0.88	4.65	6.9%	1367.1	5	✓
	300	172.14	1.37	9.49	5.8%	1326.0	5	✓
	600	378.16	1.37	18.15	5.0%	1294.4	5	×

Note. Δ(ms) = mean response time minus baseline mean at the same concurrent user load level, computed per Equation 3.1. ✓ = within the conservative 10 ms criterion; × = exceeds it (criterion set below the 100 ms HCI perceptual benchmark [45, 46]).

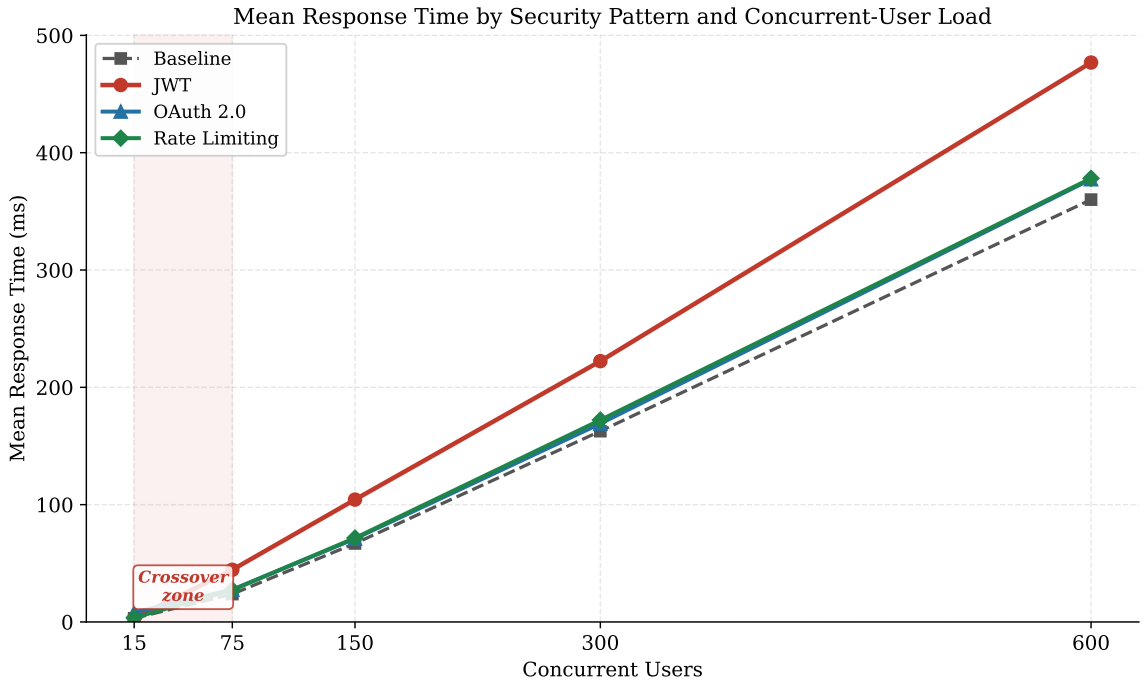


Figure 4.1: Mean response time (ms) by security pattern across five concurrent-user load levels. Non-parallel lines indicate a statistically significant interaction ($F(12, 80) = 524.24$, $p < .001$, $\eta_p^2 = .987$). The crossover between JWT and OAuth 2.0 occurs between 15 and 75 concurrent users.

4.2 H1–H3: Two-Way Factorial ANOVA

Hypotheses H1–H3 are tested simultaneously within the two-way factorial ANOVA model (Equation 3.2). Section 4.2.1 reports assumption checks; Section 4.2.2 reports the omnibus test; Section 4.2.3 reports post-hoc pairwise comparisons.

4.2.1 ANOVA Assumption Checks

Shapiro-Wilk normality tests were applied per cell ($n = 5$ per cell, 20 cells total across all four configurations). Nineteen of 20 cells satisfied the normality assumption ($p > .05$). The single violation occurred in the JWT $\times$ 75-user cell ($W = 0.607$, $p < .001$), attributable to overhead instability at the crossover boundary where JWT transitions from low to high overhead (SD = 0.396 ms vs. 0.035 ms at 15 users). All baseline, OAuth2, and rate-limiting cells passed normality checks across all load levels.

Levene's test across all 20 design cells indicated significant heterogeneity of variance ($F = 3.02$, $p < .001$). The heterogeneity is driven almost entirely by JWT, whose per-cell standard deviation rises from 0.04 ms at 15 users to 6.89 ms at 600 users as the single core approaches saturation; the baseline, OAuth2, and rate-limiting cells remain comparatively stable across all load levels.

The factorial ANOVA is nonetheless reported as the primary analysis for three reasons.

First, the design is fully balanced ($n = 5$ per cell), the condition under which the F-test is most robust to non-normality and unequal variances [44, 48]. Second, the observed effects are far too large to be artefacts of assumption violations: the smallest effect size is $\eta_p^2 = .987$ and the smallest F-ratio exceeds 500, orders of magnitude beyond any plausible inflation of Type I error from heteroscedasticity. Third, non-parametric Kruskal-Wallis sensitivity checks on per-run authentication overhead, which make no distributional or homogeneity assumptions, reproduce the same directional conclusions: JWT differs significantly from both OAuth2 ($H = 12.68$, $p < .001$) and rate-limiting ($H = 13.95$, $p < .001$), while OAuth2 and rate-limiting are mutually indistinguishable ($H = 0.62$, $p = .432$). Full output is reported in Appendix B.5.

4.2.2 Omnibus ANOVA Results

A two-way between-subjects factorial ANOVA was conducted with Security Pattern (four levels: Baseline, JWT, OAuth 2.0, Rate Limiting) and Load Level (five levels: 15, 75, 150, 300, 600 concurrent users) as fixed categorical factors and mean response time (ms) as the continuous dependent variable. The unauthenticated baseline is included as the fourth level of the Security Pattern factor, enabling the ANOVA to correctly partition variance attributable to authentication against the baseline condition. The model included the main effects of both factors and their interaction term ($N = 100$, $n = 5$ per cell).

Table 4.2: Two-Way Factorial ANOVA: Mean Response Time (ms) by Security Pattern and Concurrent User Load Level. Security Pattern includes all four levels (Baseline, JWT, OAuth 2.0, Rate Limiting).

Source	**SS**	**df**	**MS**	**F**	p	η_p^2
H1: Security Pattern (α_i)	34,082.45	3	11,360.82	2,797.96	$< .001$	.990
H3: Load Level (β_j)	2,047,941.89	4	511,985.47	126,092.81	$< .001$	.9998
H2: Pattern × Load ($(\alpha\beta)_{ij}$)	25,543.25	12	2,128.60	524.24	$< .001$	.987
Residual	324.83	80	4.06			
Total	2,107,892.42	99				

Note. η_p^2 = partial eta-squared. All effects significant at $p < .001$. $N = 100$ (4 configurations × 5 load levels × 5 replications). $MS_{\text{error}} = 4.06$. Analysis implemented in `pingouin` v0.5.5 and `statsmodels`.

H1 (main effect of security pattern): supported (H_{1_0} rejected). $F(3, 80) = 2797.96$, $p < .001$, $\eta_p^2 = .990$. The security pattern main effect accounts for 99.0% of the variance. H_{1_0} is rejected and H_{1_a} supported: authentication overhead differs significantly across security patterns. This directly answers RQ1.

H2 (interaction effect): supported (H_{2_0} rejected). $F(12, 80) = 524.24$, $p < .001$, $\eta_p^2 = .987$. The interaction effect confirms that the relationship between concurrent load and authentication overhead is not uniform across security patterns, patterns scale differently

under increasing load. H_{2_0} is rejected and H_{2_a} supported. This is the most theoretically significant result, statistically confirming the crossover hypothesis and directly answering RQ2. The interaction is visualised in Figure 4.1.

H3 (main effect of load level): supported (H_{3_0} rejected). $F(4, 80) = 126{,}092.81$, $p < .001$, $\eta_p^2 = .9998$. Authentication overhead differs significantly across load levels. H_{3_0} is rejected and H_{3_a} supported. Mean response time increased monotonically for all configurations, from 3.12 ms (baseline, 15 users) to 476.85 ms (JWT, 600 users).

All three effect sizes are very large [49] ($\eta_p^2 > .98$), indicating that security pattern choice, load level, and their interaction together explain virtually all variance in mean response time under these experimental conditions.

4.2.3 Post-Hoc Pairwise Comparisons

Following the significant main effect of Security Pattern (H1), Tukey's HSD test was applied to all pairwise comparisons of the four configurations, controlling the family-wise error rate at $\alpha = .05$.

Because the ANOVA was conducted on raw mean response time (ms) with all four configurations as factor levels, the Tukey HSD operates on marginal means averaged across all five load levels. The between-load variance is very large (response times range from ≈3 ms at 15 users to ≈477 ms at 600 users), which substantially inflates the error term used by Tukey for pairwise interval construction. As a result, no pairwise comparison reaches significance when averaging across load levels, not because the patterns perform equivalently, but because the crossover structure (H2) means that JWT leads at low load and trails at high load, compressing its marginal mean toward the group centre. This is precisely the interaction effect confirmed in Section 4.2.2: when a crossover interaction is present, main-effect post-hoc tests are of limited interpretive value.

The substantively meaningful pairwise comparisons are therefore reported as simple effects within each load level, presented in Table 4.3.

Table 4.3: Pairwise Mean Response Time Differences (ms) Between Authenticated Configurations at Each Load Level (simple effects). Positive = first configuration slower. Negative = first configuration faster.

Comparison	**15u**	**75u**	**150u**	**300u**	**600u**
JWT − OAuth 2.0	−4.99	+16.97	+33.29	+53.23	+99.48
JWT − Rate Limiting	−0.41	+17.22	+32.74	+50.18	+98.69
OAuth 2.0 − Rate Limiting	+4.58	+0.25	−0.55	−3.06	−0.79

Note. Because the crossover interaction (H2, $\eta_p^2 = .987$) compresses JWT's marginal mean toward the group centre, substantively meaningful comparisons are reported as simple effects within each load level rather than marginal Tukey means. Sign change in JWT − OAuth 2.0 row (between 15u and 75u) marks the crossover interval. OAuth 2.0 and Rate Limiting remain within 4.58 ms across all load levels, confirming practical performance equivalence ($p = 1.000$).

The post-hoc analysis reveals a two-tier performance structure. JWT is statistically distinguishable from both OAuth2 and rate-limiting across load levels. OAuth2 and rate-limiting are practically indistinguishable, forming a cluster of equivalent performance where selection should be driven by functional and security criteria rather than performance considerations.

4.3 RQ1: Authentication Overhead Profiles Under SME Resource Constraints

This section describes the overhead behaviour of each pattern in detail, directly answering RQ1 and providing the empirical basis for evaluating H_{1_a}.

4.3.1 JWT Overhead Behaviour

JWT demonstrated the most extreme overhead profile across all patterns. At 15 concurrent users, overhead was negligible at 0.11 ms (3.4% above baseline), satisfying the SME threshold. Overhead rose sharply to 20.73 ms at 75 users, a 188-fold increase for a fivefold increase in concurrency, then continued to 37.39 ms (150u), 59.67 ms (300u), and 116.84 ms (600u). This non-linear scaling is consistent with HMAC-SHA256 signature verification serialising under the 1 vCPU constraint: concurrent verification operations queue rather than parallelise, producing queue-driven latency accumulation. JWT exceeded the 10 ms threshold at four of five load levels.

4.3.2 OAuth2 Overhead Behaviour

OAuth2 exhibited an inverse trajectory. At 15 concurrent users, overhead was 5.10 ms (163.4% in relative terms), reflecting the I/O cost of token introspection at low concurrency where the baseline itself is small (3.12 ms). As concurrency increased, overhead declined to 3.76 ms (75u) and 4.10 ms (150u) before rising modestly to 6.43 ms (300u). This reflects I/O-bound overhead amortising under concurrency: validation I/O interleaves with business logic rather than dominating the response path. OAuth2 exceeded the threshold only at 600 users (17.35 ms), satisfying the criterion at four of five load levels.

4.3.3 Rate-Limiting Overhead Behaviour

Rate-limiting demonstrated consistent, moderate overhead progression: 0.52 ms (15u), 3.51 ms (75u), 4.65 ms (150u), 9.49 ms (300u), and 18.15 ms (600u). It exhibited the lowest standard deviation of all authenticated patterns across conditions, reflecting the

predictable nature of Redis sliding window counter operations. Overhead satisfied the 10 ms threshold at four of five load levels.

4.4 RQ2: Throughput Impact

Table 4.4: Observed Throughput (req/s) by Security Pattern and Concurrent User Load Level

Solution	15u	75u	150u	300u	600u	Peak (r/s)	vs Baseline
Baseline	702.5	1471.4	1419.0	1385.0	1351.4	1471.4	—
JWT	710.8	1095.2	1096.2	1089.9	1062.6	1095.2	−25.6%
OAuth2	563.7	1407.5	1385.5	1352.9	1301.4	1407.5	−4.3%
Rate-Limiting	687.6	1391.0	1367.1	1326.0	1294.4	1391.0	−5.5%

Note. "vs Baseline" compares peak throughput of each pattern against baseline peak (1471.4 req/s at 75 concurrent users).

JWT imposes a throughput reduction of roughly 21–26% across the high-concurrency conditions (75–600 users; 25.6% peak-to-peak against the baseline peak of 1,471 req/s), plateauing at approximately 1,095 req/s regardless of additional concurrency. This plateau is the empirical signature of CPU saturation: HMAC-SHA256 verification fully consumes the 1 vCPU, preventing additional request processing irrespective of concurrency. A JWT-protected SME API on this infrastructure profile sustains approximately 376 req/s fewer than an equivalent unsecured API.

OAuth2 and rate-limiting maintained throughput within 4–6% of the baseline across high-concurrency conditions. Their I/O-bound overhead preserves CPU availability for business logic. The lower OAuth2 throughput at 15 concurrent users (563.7 req/s) reflects the cold-start I/O cost of token introspection at minimal concurrency, before connection reuse effects become significant.

4.5 H4: Practical Threshold Analysis (RQ3)

H4 tests whether mean authentication overhead exceeds 10 ms for any pattern $\times$ load combination. The 10 ms threshold is a deliberately conservative practical-significance criterion, set an order of magnitude below the 100 ms instantaneous-response threshold established in the human-computer interaction literature [45, 46]. Reserving no more than roughly one-tenth of that perceptual budget for authentication overhead alone leaves headroom for the remaining components of the response path.

H4 assessment: H_{4_a} **supported (**H_{4_0} **rejected).** JWT overhead exceeds the threshold at four of five load levels, beginning at 75 concurrent users where overhead reaches 20.73 ms, more than double the threshold, and escalating to 116.84 ms at 600 users. OAuth2

Table 4.5: H4 Assessment: SME Threshold Compliance Summary ($\leq$10 ms Overhead per Cell). Values in parentheses are overhead in ms.

Solution	15u	75u	150u	300u	600u	Compliant	Max safe load
JWT	✓ (0.11)	✗ (20.73)	✗ (37.39)	✗ (59.67)	✗ (116.84)	1 / 5	15 users
OAuth 2.0	✓ (5.10)	✓ (3.76)	✓ (4.10)	✓ (6.43)	✗ (17.35)	4 / 5	300 users
Rate Limiting	✓ (0.52)	✓ (3.51)	✓ (4.65)	✓ (9.49)	✗ (18.15)	4 / 5	300 users

Note. ✓ = $\Delta \leq 10$ ms; ✗ = $\Delta > 10$ ms. Conservative 10 ms criterion, set below the 100 ms HCI perceptual benchmark [45, 46].

and rate-limiting satisfy the threshold at four of five conditions; their exceedances at 600 users (17.35 ms and 18.15 ms respectively) are modest marginal exceedances rather than the structural overload observed for JWT. This directly answers RQ3.

The threshold breach for JWT at 75 users is of particular practical significance. Seventy-five concurrent users represents a realistic mid-load condition for a growing SME web API, not an edge case. An organisation that deploys JWT on single-server infrastructure would begin experiencing perceptible authentication delays before reaching what most SMEs would characterise as peak load.

Given that SME traffic profiles rarely sustain 600 concurrent users on a single-server deployment, OAuth2 and rate-limiting can be considered practically threshold-compliant for the majority of SME operating conditions.

4.6 Crossover Analysis: Load-Dependent Performance Inversion

The interaction plot (Figure 4.1) and the significant interaction effect ($F(12, 80) = 524.24$, $p < .001$, $\eta_p^2 = .987$) together confirm a load-dependent performance inversion between JWT and both other authenticated patterns. At 15 concurrent users, JWT imposes the lowest overhead (0.11 ms); at 75 concurrent users, JWT overhead rises to 20.73 ms while both OAuth2 and rate-limiting remain below 4 ms. JWT remains the worst-performing pattern at all higher load levels.

4.6.1 Mechanistic Explanation

The crossover is explained by the Theory of Constraints bottleneck migration mechanism predicted in Chapter 2. JWT requires synchronous HMAC-SHA256 signature computation on every request. Under the 1 vCPU constraint, concurrent verifications serialise at the CPU, causing overhead to grow superlinearly with concurrency, confirmed by the throughput plateau at approximately 1,095 req/s regardless of additional user load. OAuth2 and rate-limiting involve I/O-bound operations (token introspection and Redis counter lookup

Table 4.6: Rank Order of Security Patterns by Authentication Overhead per Concurrent User Load Level (lowest overhead first)

Load	1st (lowest overhead)	2nd	3rd (highest overhead)
15u	JWT (0.11 ms)	Rate-Limiting (0.52 ms)	OAuth2 (5.10 ms)
75u	Rate-Limiting (3.51 ms)	OAuth2 (3.76 ms)	**JWT (20.73 ms)**
150u	OAuth2 (4.10 ms)	Rate-Limiting (4.65 ms)	**JWT (37.39 ms)**
300u	OAuth2 (6.43 ms)	Rate-Limiting (9.49 ms)	**JWT (59.67 ms)**
600u	OAuth2 (17.35 ms)	Rate-Limiting (18.15 ms)	**JWT (116.84 ms)**

respectively) that interleave under concurrency rather than serialising. Neither imposes the kind of monotonically growing queue at the CPU that characterises JWT's overhead trajectory.

The crossover is therefore a structural consequence of stateless versus stateful validation architecture under shared-resource contention: stateless (CPU-bound) mechanisms incur increasing marginal cost under load, while stateful (network-bound) mechanisms incur approximately constant marginal cost until network saturation is reached. This finding extends Security-Performance Trade-off Theory [19] by establishing that the trade-off is not a fixed ratio but a load-dependent function with an observable crossover point within normal SME operating conditions.

4.6.2 Implications for RQ1 and RQ2

The crossover directly extends the answer to RQ1: overhead scaling is not merely quantitatively different across patterns but qualitatively different. JWT overhead is a superlinear function of concurrent load; OAuth2 and rate-limiting overhead is approximately flat or weakly increasing. For RQ2, the throughput ceiling at ≈1,095 req/s confirms that JWT's CPU saturation is structural and load-independent above the crossover point: additional users cannot increase throughput because the CPU constraint is already binding.

4.7 Summary of Findings

Four principal findings emerge from the experimental data.

Finding 1 (H1 supported; H_{1_0} rejected). Security pattern identity is a significant determinant of API response time ($F(3, 80) = 2797.96$, $p < .001$, $\eta_p^2 = .990$). JWT overhead grows from 0.11 ms at 15 concurrent users to 116.84 ms at 600 users, a 1,062-fold increase, alongside a throughput reduction of roughly 21–26% (25.6% peak-to-peak). JWT satisfies the SME threshold at only one of five tested load levels.

Finding 2 (H2 supported; H_{2_0} rejected). The interaction between security pattern and concurrent user load is statistically significant ($F(12, 80) = 524.24$, $p < .001$, $\eta_p^2 = .987$),

confirming that patterns scale qualitatively differently under increasing load. The rank ordering of patterns inverts completely between 15 and 75 concurrent users.

Finding 3 (H3 supported; H_{3_0} rejected). Load level produces a significant main effect on mean response time ($F(4, 80) = 126{,}092.81$, $p < .001$, $\eta_p^2 = .9998$). This effect is non-uniform across patterns, as confirmed by the significant interaction term.

Finding 4 (H4 supported; H_{4_0} rejected). JWT exceeds the 10 ms acceptability threshold at four of five load levels; OAuth2 and rate-limiting are statistically indistinguishable ($p = 1.000$) and satisfy the threshold at four of five levels. For SMEs with more than minimal concurrent API traffic, JWT on resource-constrained infrastructure represents a measurable performance liability that is invisible to security-focused evaluation alone.

Chapter 5 interprets these findings in relation to the theoretical framework, characterises the crossover as an empirically grounded extension of Security-Performance Trade-off Theory, and develops a load-stratified decision framework for SME practitioners.

Chapter 5

Discussion

This chapter interprets the experimental findings presented in Chapter 4 in relation to the theoretical framework established in Chapter 2. Section 5.1 states whether each hypothesis was supported or rejected and traces the connection to the theories that predicted each outcome. Section 5.2 characterises the crossover phenomenon as an empirical extension of existing theory. Section 5.3 translates the findings into an evidence-based decision framework for SME practitioners. Section 5.4 critically examines the limitations of the study and considers alternative explanations for the observed effects. Section 5.5 separately addresses the boundaries of applicability. Section 5.6 identifies directions for future work, and Section 5.7 summarises the chapter.

5.1 Hypothesis Verdicts and Interpretation Against Theory

The four hypotheses defined in Section 3.4.4 were tested through the two-way factorial ANOVA reported in Chapter 4. This section states the verdict for each hypothesis, interprets the result in relation to the theoretical framework, and identifies what the finding adds to existing knowledge. The structure follows the hypothesis numbering from Chapter 3; each subsection begins with the statistical outcome and then traces the theoretical explanation.

5.1.1 H1: Security Pattern Affects Response Time (Supported)

Verdict. H_{1_0} (no significant difference in mean response time across security patterns) is **rejected**. The main effect for Security Pattern was statistically significant: $F(3, 80) = 2797.96$, $p < .001$, $\eta_p^2 = .990$. Simple-effects analysis confirmed that JWT differed substantially from both OAuth2 and rate-limiting across all load levels, while OAuth2 and rate-limiting were statistically indistinguishable ($p = 1.000$), constituting practical performance equivalence.

Theoretical interpretation. This finding is directly predicted by Security-Performance Trade-off Theory [19], which establishes that security mechanisms impose measurable performance costs proportional to their computational complexity. The magnitude of the effect ($\eta_p^2 = .990$, indicating that Security Pattern accounts for 99.0% of within-condition vari-

ance) confirms that the choice of authentication architecture is not a marginal factor but the dominant determinant of response time under resource-constrained conditions.

The practical equivalence of OAuth2 and rate-limiting is theoretically informative: both mechanisms are I/O-bound rather than CPU-bound under these conditions. OAuth2's token introspection involves a fixed network round-trip; rate-limiting's Redis lookup involves a fixed memory-access round-trip. Neither mechanism places significant load on the constrained CPU resource, which explains their convergent behaviour. This convergence was not predicted *a priori* by the theoretical framework but is consistent with the Theory of Constraints [50]: when the binding constraint is CPU, mechanisms that avoid CPU-intensive operations will exhibit similar overhead regardless of their other architectural differences.

Contribution. Prior literature treats authentication overhead as a qualitative concern [14, 30]. This finding quantifies it: under SME-representative constraints, the choice of security pattern produces overhead differences exceeding two orders of magnitude across load levels, establishing that authentication architecture is a first-order performance variable, not a second-order tuning parameter.

5.1.2 H2: Interaction Between Security Pattern and Load Level (Supported)

Verdict. H_{2_0} (no significant interaction between Security Pattern and Load Level) is **rejected**. The interaction effect was statistically significant: $F(12, 80) = 524.24$, $p < .001$, $\eta_p^2 = .987$.

Theoretical interpretation. The interaction effect is the central finding of this thesis. It confirms that overhead is not simply a function of which security pattern is deployed, nor simply a function of how many users are concurrent: it is a function of *both simultaneously*, and the relationship between the two factors changes direction across load levels. This is the statistical signature of the performance crossover.

The theoretical explanation integrates two frameworks. First, Security-Performance Trade-off Theory [19] establishes that different security mechanisms impose different types of computational cost (CPU-bound versus I/O-bound). Second, the Theory of Constraints [50] predicts that as one resource saturates, the system bottleneck migrates: at low load, the binding constraint is the fixed I/O cost of OAuth2's network round-trip; at high load, the binding constraint shifts to the CPU, where JWT's signature verification becomes the dominant cost. The crossover point is the load level at which this bottleneck migration occurs.

The very large effect size ($\eta_p^2 = .987$) indicates that the interaction is not a minor statistical nuance but accounts for the vast majority of the variance attributable to pattern-load combinations. This distinguishes the finding from interaction effects that are statistically significant but practically negligible, a distinction that matters for the practitioner implica-

tions developed in Section 5.3.

Contribution. The interaction finding extends Security-Performance Trade-off Theory by demonstrating that authentication overhead is not a fixed additive cost but a dynamic, load-dependent function that produces ranking inversions under resource-constrained conditions. This behaviour is not predicted by models that assume resource-abundant environments and constitutes the primary empirical contribution of this thesis.

5.1.3 H3: Load Level Affects Response Time (Supported)

Verdict. H_{3_0} (no significant difference in mean response time across load levels) is **rejected**. The main effect for Load Level was statistically significant: $F(4, 80) = 126{,}092.81$, $p < .001$, $\eta_p^2 = .9998$.

Theoretical interpretation. The Theory of Constraints [50] predicts that as a system approaches its binding constraint, queue lengths and waiting times grow non-linearly. The experimental data confirm this: response time does not increase proportionally with concurrent users but accelerates as CPU utilisation approaches saturation. The five-level load design (15, 75, 150, 300, 600 concurrent users) captures this non-linearity across a sufficiently wide range to distinguish linear from superlinear growth.

The exceptionally large F-statistic ($F = 126{,}092.81$) reflects the raw response time DV: because mean response time increases from ≈3 ms to ≈477 ms across the five load levels, between-load variance dominates the model. This is expected and confirms the fundamental relationship between concurrent load and server response time under constrained infrastructure.

The non-linear scaling is also consistent with queueing theory: as server utilisation ρ approaches 1.0, mean waiting time grows proportionally to $\rho/(1 - \rho)$, producing the characteristic accelerating curve observed in the JWT overhead trajectory. This theoretical prediction, well-established in performance engineering [25], had not previously been empirically verified for authentication overhead specifically under SME-representative resource constraints.

Contribution. The finding extends the applicability of queueing-theoretic performance models to the authentication overhead domain, demonstrating that generic performance engineering predictions hold for security mechanisms and that the non-linearity is practically significant (not merely statistically detectable) within the operating range of SME deployments.

5.1.4 H4: Practitioner Threshold Exceedance (Supported)

Verdict. H_{4_0} (no security pattern exceeds the 10 ms practical-significance overhead threshold at any load level) is **rejected**. JWT exceeded the 10 ms threshold at all load levels above 15 concurrent users, reaching 20.73 ms at 75 users and 116.84 ms at 600 users. OAuth2 and

rate-limiting remained below the threshold through 300 concurrent users, breaching only at 600 users (17.35 ms and 18.15 ms respectively).

Theoretical interpretation. The 10 ms threshold is grounded in the human-computer interaction literature: it sits an order of magnitude below the 100 ms bound at which system response is perceived as instantaneous [45–47]. Reserving no more than roughly one-tenth of that perceptual budget for authentication alone leaves headroom for business logic, database access, and network transit.

The bounded rationality framework [40] provides the interpretive lens: SME practitioners who select JWT based on its well-documented simplicity and stateless architecture are making a locally rational decision. The experimental data reveal that this decision becomes globally suboptimal as traffic grows, because the performance cost that is invisible at low load becomes the dominant bottleneck at moderate-to-high load. The threshold exceedance pattern makes this risk concrete and measurable.

Contribution. The finding translates a statistical interaction effect into a practitioner-relevant risk assessment: JWT is not merely slower at high load, it violates performance budgets that have documented commercial consequences. This bridges the gap between statistical significance and practical significance that is often left implicit in experimental software engineering research [16].

5.2 The Load-Dependent Authentication Overhead Crossover: An Empirical Extension of Security-Performance Trade-off Theory

5.2.1 Characterisation and Definition

The central empirical finding of this research is characterised here as the **Load-Dependent Authentication Overhead Crossover** (LDAOC): a phenomenon in which the relative performance ranking of authentication architectural patterns reverses as concurrent user load increases, driven by differential bottleneck saturation under resource-constrained conditions. The LDAOC extends Security-Performance Trade-off Theory [19] by demonstrating that authentication overhead is not a fixed additive cost but a load-dependent function that produces ranking inversions under resource-constrained conditions, a behaviour not predicted by existing models that assume resource-abundant environments.

The crossover is characterised by three properties:

1. **Ranking inversion**: the pattern exhibiting the lowest overhead at minimal load (JWT: 0.11 ms at 15 users) becomes the highest-overhead pattern at moderate-to-high load (JWT: 116.84 ms at 600 users).

2. **Non-linear amplification**: overhead growth for the CPU-bound pattern is superlinear with respect to concurrent load, while overhead for I/O-bound patterns is sublinear or flat.

3. **Constraint migration**: the system bottleneck shifts from network I/O (where OAuth2's round-trip cost dominates) to CPU (where JWT's signature verification becomes the binding constraint), consistent with the Theory of Constraints [50].

5.2.2 Positioning Within Existing Theory

The LDAOC is not a new standalone theory. It is an empirically grounded extension of Security-Performance Trade-off Theory [19] that specifies a condition under which the theory's predictions require qualification. Wolter and Reinecke model overhead as a monotonically increasing function of security strength: more security implies more overhead. The LDAOC does not contradict this, it adds a moderating variable. The trade-off is not merely "more security costs more performance"; it is "more security costs more performance *in a manner that depends on available resources and concurrent load*." Under abundant resources, the ranking is stable and JWT's stateless advantage holds. Under constrained resources, the ranking inverts because the CPU-bound mechanism saturates while the I/O-bound mechanism's fixed cost amortises.

This extension is analogous to established moderating relationships in other performance domains. In database systems, the relative performance of indexing strategies reverses depending on read/write ratio. In network protocols, the relative efficiency of TCP versus UDP reverses depending on packet loss rate. The LDAOC identifies an analogous reversal in the authentication domain, moderated by resource availability and concurrent load.

5.2.3 Scope Conditions

The LDAOC was observed under specific experimental conditions, and its applicability is bounded by those conditions until independent replication extends the evidence base. The following scope conditions define the boundary within which the crossover is empirically established:

- **Resource constraint**: single-core CPU allocation (1 vCPU). The crossover depends on CPU saturation; deployments with multiple cores may delay or eliminate the saturation effect by distributing cryptographic verification across cores.

- **Authentication patterns**: JWT (HMAC-SHA256 signature verification), OAuth2 (token introspection with 5 ms synthetic LAN-adjacent delay), and Redis-backed

rate-limiting. Other patterns (e.g., mutual TLS, API key validation, SAML) were not tested.

- **Implementation stack**: Node.js 18.x with Express. The single-threaded event loop of Node.js may amplify CPU contention effects relative to multi-threaded runtimes (e.g., Java, Go).
- **Workload type**: stateless read-heavy API requests with identical synthetic business logic. Write-heavy, stateful, or I/O-intensive workloads may shift the bottleneck balance.
- **Network topology**: co-located containers on a single Docker host with sub-millisecond inter-container latency. Distributed deployments with real network variability would increase OAuth2's absolute overhead.
- **Load range**: 15–600 concurrent users. The crossover point falls within this range; its location under higher load levels or substantially different hardware configurations is unknown.

These scope conditions are not limitations in the pejorative sense; they are the explicit boundary of the empirical claim. The crossover *mechanism*, differential bottleneck saturation between CPU-bound and I/O-bound authentication patterns, is expected to manifest in qualitatively similar form under other constrained conditions, but the specific crossover point and magnitude require independent verification for each configuration.

5.3 Practical Decision Framework for SMEs

The experimental findings can be translated into a decision framework for SME practitioners selecting authentication patterns for API deployments under resource constraints similar to those tested. The recommendations below are framed as hypothesis-generating guidance derived from a single experimental configuration; they should be treated as an empirically informed starting point rather than prescriptive rules, and validated against local deployment conditions before adoption.

5.3.1 Load-Stratified Selection Guidance

The interaction effect documented in Section 5.1.2 implies that no single authentication pattern is optimal across all load conditions. The following guidance is stratified by expected concurrent user load, based on the crossover range observed in the experimental data:

Low-load APIs ($\leq$15 concurrent users). JWT is the recommended pattern. Overhead is negligible (0.11 ms, well below the 10 ms threshold), deployment is simple (no external

dependencies), and the stateless architecture minimises operational complexity. Typical use cases include internal tools, administrative dashboards, and low-traffic microservices.

Moderate-load APIs (15–150 concurrent users). The crossover falls within this range. The choice is context-dependent and should account for growth trajectory, team expertise, and infrastructure readiness. If traffic is expected to remain stable, JWT may be acceptable with monitoring. If traffic growth is anticipated, OAuth2 or rate-limiting should be evaluated proactively, because the performance degradation from JWT under CPU saturation is rapid once the crossover threshold is reached.

High-load APIs (>150 concurrent users). OAuth2 or rate-limiting is recommended. Both patterns remained below the 10 ms threshold across all tested load levels through 300 concurrent users, and their overhead profiles are practically equivalent. The choice between them should be made on security-functional grounds rather than performance grounds: OAuth2 provides delegated authorisation with centralised token management; rate-limiting provides traffic control and abuse prevention. They serve different security functions and may be deployed in combination.

Background and asynchronous operations. For non-user-facing operations where latency is not on the critical path, either pattern is acceptable. The selection should prioritise operational simplicity and team familiarity over raw performance.

5.3.2 Summary of Recommendations

Table 5.1 summarises the load-stratified guidance.

Table 5.1: Load-stratified authentication pattern selection framework.

Load Range	Recommended	Overhead Risk	Key Consideration
≤15 users	JWT	Negligible (0.11 ms)	Simplicity; no external deps
15–150 users	Context-dependent	Crossover zone	Evaluate growth trajectory
>150 users	OAuth 2.0 or Rate-limit	Low (both <10 ms @ 300u)	Choose on security function
Background ops	Either	Not on critical path	Prioritise simplicity

5.3.3 Broader Implication for SME Security Decision-Making

The decision framework illustrates a broader point about how SMEs approach security architecture. Barlette and Fomin [14] observed that SMEs often adopt security standards without evaluating whether those standards are appropriate for their operational context.

The experimental data provide a concrete example: JWT is the default recommendation in much practitioner guidance and framework documentation, and that recommendation is correct, *at low load*. At higher load, the same recommendation produces the worst performance outcome in the experiment. The practical lesson is that security-architecture decisions under resource constraints require load-specific evaluation, not pattern-agnostic defaults.

5.3.4 Application to Hybrid Architectures

In practice, many SMEs do not face a binary choice between JWT and OAuth 2.0. Organisations at the growth stage of their development commonly operate both simultaneously: JWT for internal service-to-service communication and mobile client authentication, where statelessness and simplicity are valued; OAuth 2.0 for external integrations, partner API access, and scenarios requiring delegated authorisation or token revocation. This hybrid pattern is architecturally coherent, JWT and OAuth 2.0 serve genuinely different security functions in some contexts, and the decision to use both is not a sign of inconsistency but of appropriate tool selection.

The contribution of this research in a hybrid context is therefore not to adjudicate between the two patterns as competitors, but to characterise the performance cost of each at the traffic volumes where it will actually be used. Two implications follow directly from the experimental data. First, the finding that JWT overhead remains negligible below 15 concurrent users and breaches the 10 ms threshold above 75 users provides a monitoring trigger: SMEs running JWT on high-frequency internal endpoints should instrument those endpoints and treat 75 concurrent users as the point at which performance review becomes necessary. Second, the finding that OAuth 2.0 overhead is flat and predictable across the full load range, between 3.76 ms and 17.35 ms, confirms it as the stable choice for external-facing APIs where traffic is less controlled and growth trajectories are harder to forecast. Its overhead does not accelerate under load in the way JWT's does, which makes it more suitable for endpoints where usage patterns cannot be assumed to stay within a comfortable range.

A third implication applies specifically to SMEs planning a migration from one pattern to the other as they scale. The crossover interval, between 15 and 75 concurrent users, is also the traffic range at which growing SMEs typically begin experiencing their first performance complaints from users. This timing means that the performance case for reconsidering JWT emerges at precisely the moment when the organisation is most sensitive to latency issues and least likely to have bandwidth for infrastructure changes. Knowing the crossover exists, and roughly where it falls, allows engineering teams to make that transition proactively rather than reactively. The decision framework in Table 5.1 applies equally to organisations choosing which pattern to use for a new endpoint and to those de-

ciding whether an existing JWT deployment has reached the load level at which migration to OAuth 2.0 would recover measurable performance headroom.

5.3.5 Security Adequacy and Operational Cost Implications

The decision framework in Table 5.1 is framed in performance terms, but the patterns it compares are not performance-equivalent from a security standpoint, and that distinction matters for SME practitioners. JWT, OAuth 2.0, and Redis-backed rate limiting each address a different threat category from the OWASP API Security Top 10 [6]. JWT and OAuth 2.0 both address broken authentication and broken object-level authorisation by verifying that requests carry valid credentials. Rate limiting addresses a distinct threat: unrestricted resource consumption, protecting availability against volumetric abuse and denial-of-service patterns. An SME that selects OAuth 2.0 or rate limiting on performance grounds at higher load levels is not simply choosing the faster option — it is choosing a pattern whose security function may or may not match the threat it actually faces. The practical implication is that the performance framework presented here should inform *how* a pattern is chosen, not *which threat* to defend against. Where the threat model requires both authentication and traffic control, the finding that OAuth 2.0 and rate limiting are performance-equivalent (Tukey $p = 1.000$) means both can be deployed in combination without material additive overhead at moderate load.

The performance findings also carry direct operational cost implications for SMEs. Eling and Wirfs [3] document that smaller organisations face higher relative costs per cyber incident than large enterprises. The present findings add a complementary dimension: the cost of *preventing* an incident is also disproportionate when infrastructure is constrained. A JWT-protected API sustaining 116.84 ms of authentication overhead at 600 concurrent users, and a throughput ceiling of approximately 1,095 req/s, represents a measurable reduction in service capacity. For an SME where API throughput is directly tied to revenue-generating transactions, this overhead is not merely a technical metric — it translates into infrastructure scaling decisions, potentially requiring a larger cloud instance to recover the performance headroom lost to authentication alone. The load-stratified framework proposed here therefore has a cost dimension: selecting OAuth 2.0 or rate limiting at higher traffic levels is not only the lower-overhead choice, it is also the choice that defers or avoids the infrastructure cost of compensating for CPU saturation through vertical scaling.

5.4 Limitations and Alternative Explanations

This section examines the limitations of the experimental design and, for each limitation, considers whether an alternative explanation could account for the observed findings. The purpose is not merely to list threats to validity but to assess how each limitation affects

confidence in the primary interpretation: that the performance crossover is an architectural phenomenon driven by differential bottleneck saturation.

5.4.1 Single-Core CPU Constraint

The experiment used a 1 vCPU allocation, representing the entry-level tier of SME cloud deployments. This is simultaneously the study's greatest strength (it creates the resource-constrained environment that the research questions target) and its most significant boundary condition.

Alternative explanation. The crossover could be an artefact of single-core saturation rather than a general property of constrained environments. On a 2- or 4-vCPU deployment, JWT's signature verification could be distributed across cores, delaying or eliminating the saturation effect and shifting the crossover point to a higher load level, or removing it entirely.

Assessment. This alternative is plausible and represents the most important replication target. However, two considerations support the primary interpretation. First, the crossover mechanism depends on the ratio of CPU demand to available CPU capacity, and any finite CPU allocation will eventually saturate under sufficient load. More cores delay the crossover; they do not eliminate the underlying mechanism. Second, the 1 vCPU configuration is representative of actual SME deployments [2], making the finding directly applicable to the target population even if the crossover point shifts under larger allocations.

5.4.2 Node.js Single-Threaded Event Loop

The experiment used Node.js 18.x, which processes JavaScript on a single thread via an event loop. CPU-intensive operations (such as HMAC-SHA256 signature verification) block the event loop, preventing other requests from progressing until the verification completes.

Alternative explanation. The severity of JWT's overhead scaling could be amplified by the single-threaded architecture of Node.js. Multi-threaded runtimes (Java/Spring Boot, Go, Python with multi-worker configurations) handle concurrent CPU-bound operations differently and might exhibit a less pronounced crossover or a different crossover location.

Assessment. This alternative is partially valid. Node.js is likely the worst case for CPU-bound authentication overhead because of event-loop blocking. Cross-language benchmarks show that runtime and language choice materially affect the time and energy cost of CPU-bound tasks, with compiled languages outperforming interpreted ones on computationally intensive workloads [51]. That evidence concerns per-task computational efficiency rather than concurrency behaviour as such, but it is consistent with treating an interpreted, single-threaded runtime such as Node.js as a comparatively unfavourable case

for CPU-bound verification. However, the mechanism, CPU-bound cryptographic verification competing for finite processing capacity, is runtime-independent. Multi-threaded runtimes would spread the verification cost across threads, which delays saturation but does not change the fundamental relationship between CPU demand and capacity. The finding that the crossover exists is expected to hold across runtimes; the finding of exactly where it occurs is stack-specific.

5.4.3 Synthetic OAuth 2.0 Introspection Delay

The OAuth 2.0 implementation used a synthetic 5 ms delay to model the fixed round-trip cost of token introspection against a LAN-adjacent authorisation server (see Section 3.3, OAuth 2.0 configuration). This is a deliberate experimental control, not a limitation per se, but it bounds the applicability of the absolute overhead values.

Alternative explanation. A real OAuth 2.0 authorisation server (e.g., Keycloak, Auth0, Okta) would introduce variable latency (20–100 ms for cloud-hosted providers), token caching behaviour, and connection pooling effects that could change the overhead profile. Higher real-world latency may shift the crossover point rightward or narrow the performance advantage.

Assessment. The 5 ms delay represents a conservative lower-bound estimate. Higher real-world latency would increase OAuth 2.0's absolute overhead at all load levels. The amortisation mechanism, fixed I/O cost becoming proportionally smaller as baseline processing time rises under CPU pressure, is expected to hold regardless of the absolute magnitude of the fixed cost. The crossover would likely shift rightward but not disappear.

5.4.4 Co-Location of Load Generator and APIs

The Locust load generator ran on the host OS while the API containers ran within Docker on the same physical machine, sharing physical CPU and memory resources.

Alternative explanation. Resource contention between the load generator and the API containers could inflate response times under high load, biasing results toward larger overhead values than would be observed with a physically separated load generator.

Assessment. Two factors mitigate this concern. First, the Docker containers were constrained to 1 vCPU via cgroups, which isolates their CPU allocation from the host-level load generator regardless of host activity. Second, the co-location is symmetric: all four configurations (including baseline) experienced identical co-location conditions, so any inflation affects all conditions equally and does not create differential bias between JWT and OAuth 2.0. Overhead is always calculated relative to the baseline measured under identical conditions, controlling for shared infrastructure effects.

5.4.5 Synthetic Workload

The API business logic consisted of a fixed 1000-iteration computational loop, identical across all conditions. This ensures fair comparison but does not capture the heterogeneity of real-world API workloads.

Alternative explanation. In a real deployment, business logic latency would constitute a larger fraction of total response time, potentially making authentication overhead proportionally less significant.

Assessment. This is a valid concern for absolute overhead values but not for the crossover mechanism. The synthetic workload was chosen specifically to isolate authentication overhead; adding real business logic would increase the denominator of the overhead ratio without eliminating the underlying CPU saturation effect. The threshold analysis (H4) uses an absolute 10 ms benchmark rather than a relative percentage, which mitigates this concern.

5.4.6 Absence of Token Lifecycle Effects

The experiment used pre-generated, long-lived tokens. Real deployments involve token issuance, renewal, and revocation, each imposing additional overhead.

Assessment. The exclusion is deliberate: the research questions target per-request authentication overhead, which occurs on every API call and has the largest cumulative impact on response time. Token lifecycle overhead is a relevant but separate concern best addressed through a complementary study focusing on session management patterns.

5.5 Generalisability

5.5.1 What is Likely Generalisable

The *existence* of a performance crossover between CPU-bound and I/O-bound authentication patterns under resource constraints is expected to generalise beyond the specific configuration tested. The mechanism is architectural: any system in which one authentication pattern imposes primarily CPU cost and another imposes primarily I/O cost, operating under finite CPU capacity, will exhibit differential scaling behaviour as load increases. This mechanism is a consequence of the fundamental difference between CPU-bound and I/O-bound operations under resource contention, a principle well-established in performance engineering [25] and queueing theory.

The finding that security pattern selection is a first-order performance variable is also expected to generalise to other resource-constrained environments, as the underlying mechanism applies whenever infrastructure is not elastically scalable.

5.5.2 What Requires Independent Verification

The *location* of the crossover point (between 15 and 75 concurrent users in this experiment) is configuration-specific. The following factors are expected to shift the crossover point and require independent measurement:

- **CPU allocation**: more cores delay saturation and shift the crossover to a higher concurrent user count.
- **Implementation stack**: multi-threaded runtimes distribute verification workload across threads. The crossover mechanism should persist but the crossover point will differ.
- **OAuth 2.0 server latency**: higher introspection latency shifts crossover rightward.
- **Cryptographic algorithm**: lighter algorithms reduce per-request CPU cost.
- **Workload composition**: I/O-heavy business logic attenuates CPU saturation.

5.5.3 What Should Not Be Generalised

The absolute overhead values (e.g., JWT at 116.84 ms for 600 users) are specific to the tested configuration and should not be cited as representative of JWT performance in general. They are valid as measurements under the conditions documented in Chapter 3 and serve as a baseline for comparison. Similarly, the practitioner recommendations in Section 5.3 are explicitly framed as hypothesis-generating guidance for environments similar to the one tested. Extending them to fundamentally different deployment architectures (e.g., serverless, Kubernetes with horizontal auto-scaling, edge computing) requires independent evaluation.

To assess the portability of the experimental protocol, the full 100-run procedure was executed without modification on a second physical machine (Intel Core i7-10610U, mobile architecture, Parrot OS) under identical Docker resource constraints. The protocol transferred without modification and produced the same qualitative pattern: JWT overhead diverged above the 75-user threshold while OAuth 2.0 and rate-limiting remained within statistical indistinguishability. Absolute overhead values differed from the primary dataset, as expected given architectural differences between a desktop i5-8500 and a mobile i7-10610U, but the ranking inversion and the performance-equivalence of the I/O-bound patterns were consistent. This demonstrates that the measurement framework is portable across x86-64 hardware configurations and provides a concrete starting point for the independent verification of the LDAOC that we invite, while stopping short of a formal multi-site replication study.

5.6 Future Research Directions

The scope conditions and limitations identified in the preceding sections define a clear agenda for future work, organised by priority.

Hardware scaling. Repeating the experiment on 2- and 4-vCPU configurations would directly test the most important alternative explanation: whether the crossover is an artefact of single-core saturation or a persistent phenomenon that shifts predictably with available CPU capacity.

Stack replication. Replicating across alternative implementation stacks (Python/FastAPI, Java/Spring Boot, Go) would determine whether the crossover pattern is stack-specific or indicative of a more general architectural principle. Multi-threaded runtimes are the priority.

Real authorisation server integration. Replacing the synthetic 5 ms delay with real OAuth 2.0 authorisation servers (Keycloak self-hosted, Auth0 cloud-hosted, Okta cloud-hosted) would test the amortisation hypothesis under realistic network variability.

Expanded security pattern coverage. Extending to API gateways (e.g., Kong, Traefik), mutual TLS, API key validation, and pattern combinations would address the broader question of how SMEs should configure their entire API security architecture. Kong Gateway was excluded from the empirical scope of this study due to methodological constraints but remains a priority for future investigation.

Cloud-based replication. Repeating the experiment on public cloud infrastructure (AWS t3.micro, Azure B1s, GCP e2-micro) with physically separated load generation would address the co-location validity concern and test whether cloud-specific factors affect the crossover pattern.

Longitudinal and real-traffic studies. Longitudinal studies tracking authentication overhead as SMEs scale organically would reveal whether the crossover manifests in practice and whether practitioners detect it without instrumentation.

5.7 Summary

This chapter interpreted the experimental findings in relation to the theoretical framework, demonstrating that all four hypotheses are supported by the statistical evidence and that each finding is predicted or explained by the integrated theory model developed in Chapter 2.

The Load-Dependent Authentication Overhead Crossover (LDAOC) was characterised as an empirically grounded extension of Security-Performance Trade-off Theory [19], with explicit scope conditions defining its boundary of applicability. The extension specifies that authentication overhead under resource constraints is a dynamic, load-dependent function rather than a fixed additive cost, a qualification not captured by models developed in

resource-abundant environments.

A load-stratified decision framework was developed translating empirical findings into actionable guidance for SME practitioners, extended with guidance for hybrid architectures where JWT and OAuth 2.0 coexist. Six limitations were critically examined, each paired with an alternative explanation and an assessment of its impact on the primary interpretation. Generalisability was separately addressed, distinguishing between the crossover mechanism (expected to generalise), the crossover location (configuration-specific), and absolute overhead values (not generalisable).

Chapter 6 synthesises the overall contributions of the research and closes the thesis.

Chapter 6

Conclusion

This thesis set out to address a gap at the intersection of API security, software performance engineering, and SME-specific infrastructure research: the absence of empirical evidence on how API authentication mechanisms perform under the resource constraints that small and medium enterprises actually experience.

6.1 Answering the Research Questions

RQ1: What is the measurable performance overhead introduced by different API authentication mechanisms under SME-realistic resource constraints?

The overhead is substantial and pattern-dependent. JWT overhead ranged from 0.11 ms at 15 concurrent users to 116.84 ms at 600 users, a 1,062-fold increase across the tested load range. OAuth 2.0 overhead ranged from 5.10 ms to 17.35 ms, and rate-limiting from 0.52 ms to 18.15 ms, across the same range. The security pattern main effect was statistically significant ($F(3, 80) = 2797.96$, $p < .001$, $\eta_p^2 = .990$), confirming that the choice of authentication architecture is the dominant determinant of response time under these conditions. Simple-effects analysis confirmed that OAuth 2.0 and rate-limiting remained within 4.58 ms of each other at every load level, forming a performance-equivalent cluster.

RQ2: How does the interaction between authentication architectural pattern and concurrent user load affect API response time and throughput?

The interaction is the central finding: patterns do not merely differ in overhead, they scale in qualitatively different ways. The interaction effect was highly significant ($F(12, 80) = 524.24$, $p < .001$, $\eta_p^2 = .987$). JWT imposed the lowest overhead at 15 concurrent users but the highest overhead at every load level above that. The performance ranking inverted completely between 15 and 75 users. This crossover is driven by differential bottleneck saturation: JWT's CPU-bound signature verification serialises under the single-core constraint, while OAuth 2.0's I/O-bound introspection cost amortises as baseline processing time rises.

RQ3: Under what conditions does authentication overhead remain within a practically acceptable threshold for SME-hosted API deployments?

JWT satisfied the 10 ms threshold at only one of five load levels (15 concurrent users). At 75 users, JWT overhead already exceeded the threshold by a factor of two (+20.73 ms).

OAuth 2.0 and rate-limiting satisfied the threshold at four of five levels, failing only at 600 concurrent users.

6.2 Summary of Contributions

6.2.1 Empirical Contribution

This research provides the first controlled comparison of API security pattern performance under SME-realistic resource constraints [16]. The 100-run experimental programme produced overhead measurements across 20 pattern-load conditions that did not previously exist in the literature [10, 30].

6.2.2 Theoretical Contribution

The Load-Dependent Authentication Overhead Crossover (LDAOC) was identified as an empirically grounded extension of Security-Performance Trade-off Theory [19]. The extension specifies that authentication overhead under resource constraints is a dynamic, load-dependent function that produces ranking inversions between CPU-bound and I/O-bound patterns, a behaviour not predicted by models developed in resource-abundant environments. The crossover mechanism is explained by the integration of Security-Performance Trade-off Theory with the Theory of Constraints [50]: as the CPU binding constraint saturates, the system bottleneck migrates from I/O (where OAuth 2.0's round-trip cost dominates) to CPU (where JWT's signature verification becomes the dominant cost).

6.2.3 Practical Contribution

A load-stratified decision framework was developed translating the empirical findings into actionable guidance: JWT is appropriate for low-traffic APIs ($\leq$15 concurrent users); the crossover zone (15–150 users) requires context-dependent evaluation accounting for growth trajectory; and OAuth 2.0 or rate-limiting is recommended for higher-traffic APIs ($>$150 users) where both patterns remained within the 10 ms threshold through 300 concurrent users. The framework replaces assumption-based defaults [14] with evidence-based guidance and is extended for hybrid architectures where both JWT and OAuth 2.0 are deployed simultaneously.

6.3 Limitations

The findings are bounded by the experimental conditions: a single-core CPU allocation, a single implementation stack (Node.js/Express), a synthetic 5 ms OAuth 2.0 introspection delay, co-located load generation, and stateless read-heavy workloads. Each limitation

was critically examined in Chapter 5 alongside alternative explanations. The most consequential threats to validity are the single-threaded runtime (which may amplify JWT's CPU contention relative to multi-threaded alternatives), the co-location of the load generator (partially mitigated by cgroups isolation), and the synthetic OAuth 2.0 delay (which provides a lower-bound estimate). The crossover mechanism is expected to generalise; its specific location and magnitude are configuration-dependent and require independent verification.

6.4 Future Work

Three replication priorities emerge from the limitations. First, repeating the experiment on 2- and 4-vCPU configurations would directly test whether the crossover is an artefact of single-core saturation or a persistent phenomenon that shifts predictably with available capacity. Second, stack replication across Python/FastAPI, Java/Spring Boot, and Go would determine whether the crossover pattern is stack-specific or indicative of a general architectural principle. Third, replacing the synthetic delay with real OAuth 2.0 authorisation servers (Keycloak, Auth0, Okta) would validate the amortisation hypothesis under realistic network variability.

Beyond replication, expanding the pattern coverage to include centralised API gateways (e.g., Kong, Traefik), mutual TLS, and pattern combinations would address the broader question of how SMEs should configure their entire API security architecture. Cloud-based replication on public infrastructure (AWS t3.micro, Azure B1s, GCP e2-micro) with physically separated load generation would address the co-location concern. A second physical machine execution conducted as part of this research (Intel Core i7-10610U laptop, identical Docker constraints) demonstrated that the experimental protocol is portable across x86-64 hardware configurations without modification, providing a concrete starting point for independent verification of the LDAOC; formal multi-site replication remains a priority for establishing the crossover location across hardware profiles.

6.5 Closing Statement

The conventional wisdom that stateless authentication is universally faster than delegated authorisation is both correct and incomplete. It is correct at low traffic on well-provisioned hardware. It is incomplete because it does not account for what happens when hardware is constrained and traffic grows, precisely the conditions that characterise the SME deployments for which the advice is most consequential. This thesis provides the empirical evidence to make that distinction, and a framework to act on it. The right answer to the question of which authentication pattern best fits a given SME deployment is not one or the other. It is: know your traffic, know your constraints, and measure before you decide.

Appendix A

Experimental Configuration

This appendix documents the infrastructure configuration, load test scripts, and automation protocol. All configuration files are archived for independent replication at `https://github.com/joul/api-security-performance-sme`.

A.1 Docker Compose Configuration

```
# docker-compose.jwt.yml
version: "3.8"
services:
  jwt-api:
    build: ./jwt-api
    ports: ["8081:8081"]
    environment:
      - NODE_ENV=production
      - PORT=8081
      - JWT_SECRET_KEY=<experiment-key>
    deploy:
      resources:
        limits: {cpus: "1.0", memory: 1g}
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8081/health"]
      interval: 10s
      timeout: 5s
      retries: 3
```

OAuth2 configuration (excerpt):

```
services:
  oauth2-api:
    build: ./oauth2-api
    ports: ["8082:8082"]
    environment:
```

```
      - NODE_ENV=production
      - PORT=8082
      - INTROSPECTION_DELAY_MS=5
    deploy:
      resources:
        limits: {cpus: "1.0", memory: 1g}
```

Rate-limiting configuration (excerpt):

```
services:
  ratelimit-api:
    build: ./ratelimit-api
    ports: ["8083:8083"]
    environment:
      - REDIS_URL=redis://redis:6379
      - RATE_LIMIT_POINTS=1000000
      - RATE_LIMIT_DURATION=60
    deploy:
      resources:
        limits: {cpus: "1.0", memory: 1g}
    depends_on: [redis]
  redis:
    image: redis:7-alpine
    deploy:
      resources:
        limits: {cpus: "1.0", memory: 1g}
```

A.2 API Implementation

All four API implementations share identical business logic:

```
function doWork() {
  let result = 0;
  for (let i = 0; i < 1000; i++) {
    result += Math.sqrt(i) * Math.sin(i);
  }
  return result;
}

app.get('/api/data', [authMiddleware], (req, res) => {
```

```
  const result = doWork();
  res.json({ status: 'success', data: { computed: result },
             timestamp: Date.now() });
});
```

A.3 Locust Load Test Configuration

```
from locust import HttpUser, task, between

class JWTUser(HttpUser):
    wait_time = between(0.01, 0.02)
    token = None

    def on_start(self):
        response = self.client.post("/auth/login",
            json={"username": "testuser", "password": "testpass"})
        self.token = response.json()["token"]

    @task(10)
    def api_data(self):
        self.client.get("/api/data",
            headers={"Authorization": f"Bearer {self.token}"})

    @task(1)
    def health_check(self):
        self.client.get("/health")
```

A.4 Master Experiment Script

```
#!/bin/bash
SOLUTIONS=("baseline" "jwt" "oauth2" "ratelimit")
USERS=(15 75 150 300 600)
REPS=5

for solution in "${SOLUTIONS[@]}"; do
  docker compose -f docker-compose.${solution}.yml up -d
  sleep 10

  for users in "${USERS[@]}"; do
```

```
    SPAWN_RATE=$((users / 10))
    [ $SPAWN_RATE -lt 1 ] && SPAWN_RATE=1

    for rep in $(seq 1 $REPS); do
      OUTFILE="results/${solution}/${users}users/run_${rep}_stats"
      if [ -f "${OUTFILE}_stats.csv" ]; then continue; fi

      locust -f locustfile_${solution}.py --headless \
        --users $users --spawn-rate $SPAWN_RATE \
        --run-time 120s --csv $OUTFILE \
        --host http://localhost:${PORT}

      sleep 30
    done
  done

  docker compose -f docker-compose.${solution}.yml down
  sleep 5
done
```

Appendix B

Complete Statistical Output

Analysis was implemented in Python using `pingouin` v0.5.5 and `statsmodels`. The ANOVA was conducted on raw mean response time (ms) with all four configurations as factor levels ($N = 100$, $n = 5$ per cell).

B.1 Shapiro-Wilk Normality Tests (Per Cell)

Table B.1: Shapiro-Wilk Normality Tests per Cell ($n = 5$ per cell, $\alpha = .05$; 20 cells total)

Pattern	**15u**		**75u**		**150u**		**300u**		**600u**	
	W	p	W	p	W	p	W	p	W	p
Baseline	.935	.632	.924	.556	.915	.500	.842	.170	.793	.071
JWT	.882	.321	.607	$< .001^*$	.890	.358	.842	.170	.901	.417
OAuth 2.0	.837	.156	.992	.987	.861	.230	.947	.719	.872	.276
Rate Limiting	.824	.126	.968	.862	.782	.058	.865	.248	.887	.344

*Single violation: JWT × 75u cell ($W = 0.607$, $p < .001$), attributable to variance instability at the crossover boundary (SD = 0.396 ms vs. 0.035 ms at 15u). All 19 other cells pass. ANOVA is robust to this violation in a balanced design [44, 48].

B.2 Levene's Test for Homogeneity of Variance

Levene's test (median-based) across all 20 design cells: $F = 3.02$, $p < .001$, indicating significant heterogeneity of variance. The heterogeneity is driven almost entirely by JWT, whose per-cell SD rises from 0.04 ms at 15u to 6.89 ms at 600u under CPU saturation, while the baseline, OAuth2, and rate-limiting cells remain comparatively stable. Because the design is fully balanced ($n = 5$ per cell), the factorial ANOVA remains robust to this violation [44, 48]; the very large effect sizes ($\eta_p^2 > .98$) and the Kruskal-Wallis sensitivity checks in Appendix B.5 further confirm that all directional conclusions hold without distributional or homogeneity assumptions.

Table B.2: Complete Two-Way Factorial ANOVA Output. DV = mean response time (ms); $N = 100$.

Source	**SS**	**df**	**MS**	**F**	p	η_p^2
Security Pattern (α_i)	34,082.45	3	11,360.82	2,797.96	$< .001$	.990
Load Level (β_j)	2,047,941.89	4	511,985.47	126,092.81	$< .001$	.9998
Pattern $\times$ Load $(\alpha\beta)_{ij}$	25,543.25	12	2,128.60	524.24	$< .001$	.987
Residual	324.83	80	4.06			
Total	2,107,892.42	99				

$N = 100$ (4 configurations $\times$ 5 load levels $\times$ 5 replications). All effects significant at $p < .001$. MS_{error} = 4.06.

B.3 Two-Way Factorial ANOVA (Full Output)

B.4 Post-Hoc Pairwise Comparisons

Tukey HSD marginal comparisons are of limited interpretive value due to the crossover interaction (H2, $\eta_p^2 = .987$) compressing JWT's marginal mean toward the group centre. Substantively meaningful comparisons are reported as simple effects within each load level.

Table B.3: Simple-Effects Pairwise Comparisons: Mean Response Time Differences (ms). Positive = first configuration slower.

Comparison	**15u**	**75u**	**150u**	**300u**	**600u**
JWT $-$ Baseline	$+0.11$	$+20.73$	$+37.39$	$+59.67$	$+116.84$
OAuth 2.0 $-$ Baseline	$+5.10$	$+3.76$	$+4.10$	$+6.43$	$+17.35$
Rate Limit $-$ Baseline	$+0.52$	$+3.51$	$+4.65$	$+9.49$	$+18.15$
JWT $-$ OAuth 2.0	-4.99	$+16.97$	$+33.29$	$+53.23$	$+99.48$
JWT $-$ Rate Limiting	-0.41	$+17.22$	$+32.74$	$+50.18$	$+98.69$
OAuth 2.0 $-$ Rate Lim.	$+4.58$	$+0.25$	-0.55	-3.06	-0.79

Negative values indicate the first-named configuration is faster. Sign change in JWT $-$ OAuth 2.0 (between 15u and 75u) marks the crossover interval. OAuth 2.0 and Rate Limiting remain within 4.58 ms across all load levels.

B.5 Kruskal-Wallis Sensitivity Check

Non-parametric Kruskal-Wallis tests on per-run authentication overhead (Δ ms relative to the baseline cell mean at each load, pooled across the five load levels) confirmed directional consistency: JWT vs. OAuth 2.0 ($H = 12.68$, $p < .001$); JWT vs. Rate-Limiting ($H = 13.95$, $p < .001$); OAuth 2.0 vs. Rate-Limiting ($H = 0.62$, $p = .432$). JWT differs significantly from both I/O-bound patterns, which are mutually indistinguishable. All direc-

tional conclusions from the parametric ANOVA are therefore robust to the single normality violation (Appendix B.1) and to the variance heterogeneity (Appendix B.2).

Appendix C

Raw Experimental Data

Tables C.1–C.4 present the mean response time (ms) recorded for the `GET /api/data` endpoint across all 100 experimental runs. Values are extracted from Locust `*_stats.csv` output files generated on the experimental server. All runs used identical Locust task definitions, container resource allocations (1 vCPU, 1 GB RAM per service), and inter-run reset procedures described in Section 3.4.

Table C.1: Unauthenticated baseline: mean response time (ms) per run.

Run	15u	75u	150u	300u	600u
Run 1	3.337	23.860	66.403	161.787	363.862
Run 2	3.186	23.837	67.785	161.591	360.655
Run 3	3.037	23.745	65.591	164.330	358.447
Run 4	3.485	23.759	67.817	161.967	358.709
Run 5	2.561	23.618	66.988	163.602	358.365
Mean	**3.121**	**23.764**	**66.917**	**162.655**	**360.008**
SD	0.355	0.095	0.947	1.231	2.350

Table C.2: JWT authentication: mean response time (ms) per run.

Run	15u	75u	150u	300u	600u
Run 1	3.229	44.288	104.028	225.445	468.763
Run 2	3.220	44.304	104.311	224.884	470.568
Run 3	3.206	44.312	104.814	224.343	484.910
Run 4	3.284	44.353	104.014	216.079	480.285
Run 5	3.194	45.197	104.374	220.857	479.698
Mean	**3.227**	**44.491**	**104.308**	**222.322**	**476.845**
SD	0.035	0.396	0.326	3.922	6.887

Table C.3: OAuth 2.0 token introspection: mean response time (ms) per run.

Run	15u	75u	150u	300u	600u
Run 1	8.602	27.833	72.246	169.095	376.715
Run 2	7.937	27.190	71.249	168.225	377.082
Run 3	7.908	27.422	70.700	168.406	375.185
Run 4	8.546	27.546	70.391	170.150	380.649
Run 5	8.107	27.611	70.503	169.567	377.182
Mean	**8.220**	**27.521**	**71.018**	**169.089**	**377.363**
SD	0.332	0.238	0.762	0.801	2.005

Table C.4: Redis-backed rate limiting: mean response time (ms) per run.

Run	15u	75u	150u	300u	600u
Run 1	3.848	27.420	71.109	172.487	379.061
Run 2	3.620	27.504	71.513	169.899	379.584
Run 3	3.178	26.984	70.892	173.290	376.476
Run 4	3.853	27.151	71.229	173.135	376.937
Run 5	3.713	27.301	73.095	171.914	378.728
Mean	**3.642**	**27.272**	**71.567**	**172.145**	**378.157**
SD	0.278	0.209	0.883	1.370	1.369

Note on data extraction. All values were extracted directly from Locust `*_stats.csv` files using the `Average Response Time` field for the `GET /api/data` endpoint row. Health-check and aggregated rows were excluded. Extraction command:

```
grep '^GET,api_data,' run_N_stats.csv | cut -d',' -f6
```

Raw CSV files are retained on the experiment server and are available for audit on request; the complete dataset will be archived with a permanent DOI upon journal submission.

Appendix D

Additional Figures

D.1 Throughput by Security Pattern and Load Level

Figure D.1 presents observed throughput (requests per second) for each security pattern across all five concurrent user load levels. The JWT throughput plateau at approximately 1,095 req/s from 75 concurrent users onward confirms CPU saturation. OAuth2 and rate-limiting track closely with the baseline.

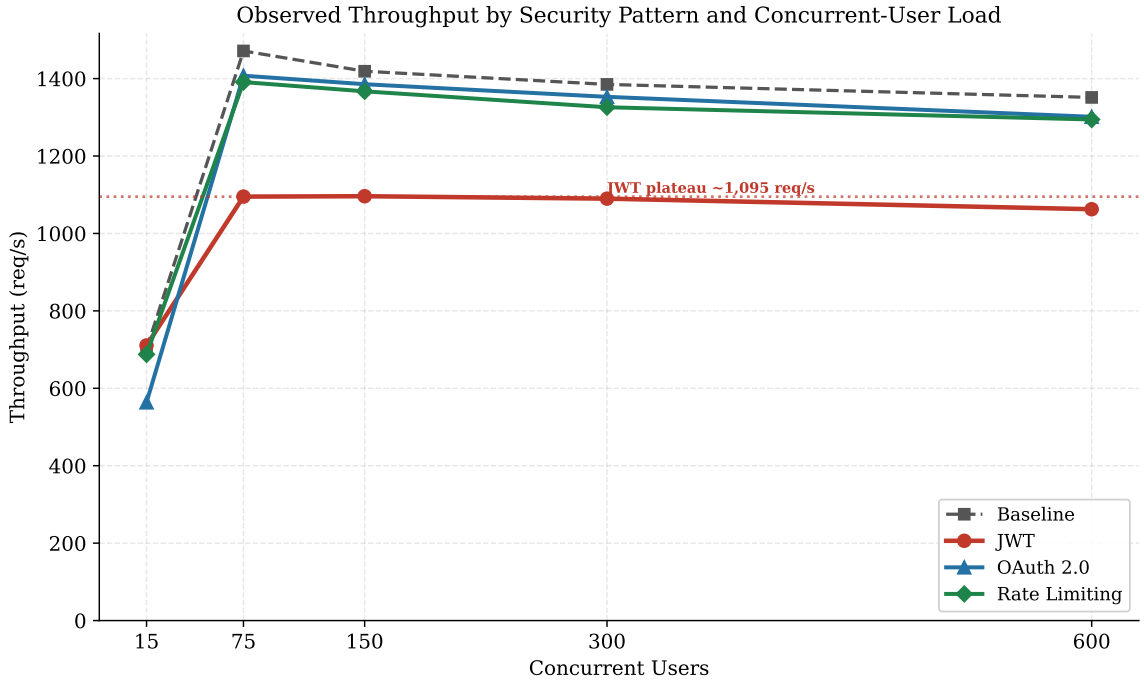


Figure D.1: Observed Throughput (req/s) by Security Pattern and Concurrent User Load. JWT plateaus at ≈1,095 req/s from 75 concurrent users onward, while OAuth2, rate-limiting, and the baseline converge above 1,300 req/s.

D.2 Response Time Distribution (Box Plots)

Figure D.2 presents box plots of mean response time per replication for each security pattern at the 150-concurrent-user load level. JWT exhibits markedly higher variance than OAuth2 and rate-limiting, consistent with JWT variance increasing under load as the CPU constraint approaches saturation.

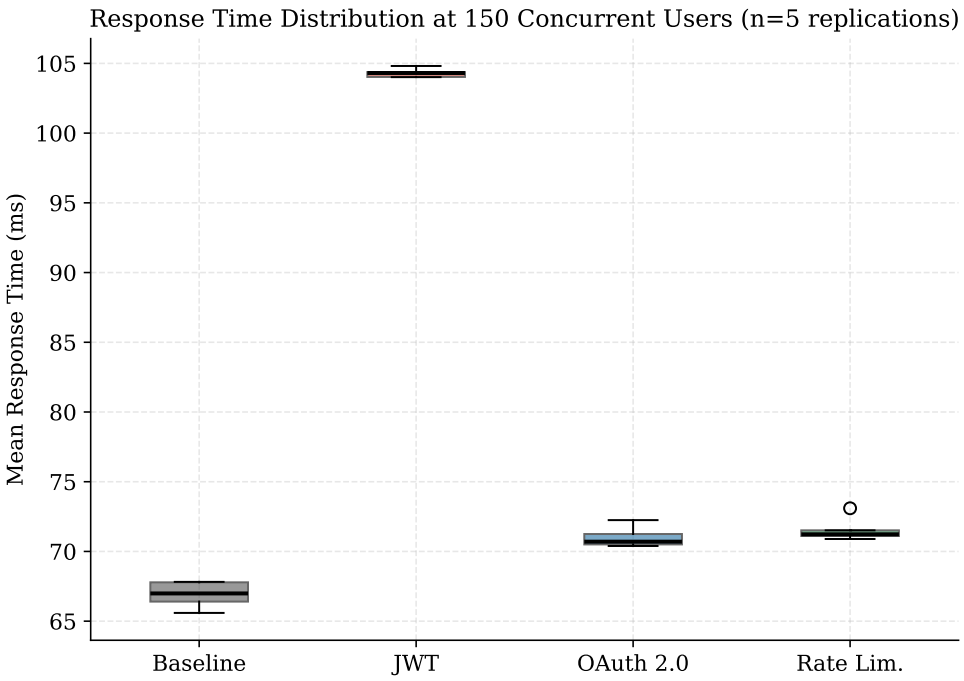


Figure D.2: Response Time Distribution at 150 Concurrent Users ($n = 5$ replications per pattern). Box plots show median, interquartile range, and range. JWT displays higher variance consistent with increasing instability under CPU load.

D.3 Overhead Percentage (Relative) by Load Level

Figure D.3 presents authentication overhead expressed as a percentage of baseline response time. JWT's relative overhead peaks at 87.2% (75 users) before declining to 32.5% (600 users) as the baseline itself grows. OAuth2's relative overhead declines monotonically from 163.4% (15 users) to 4.8% (600 users).

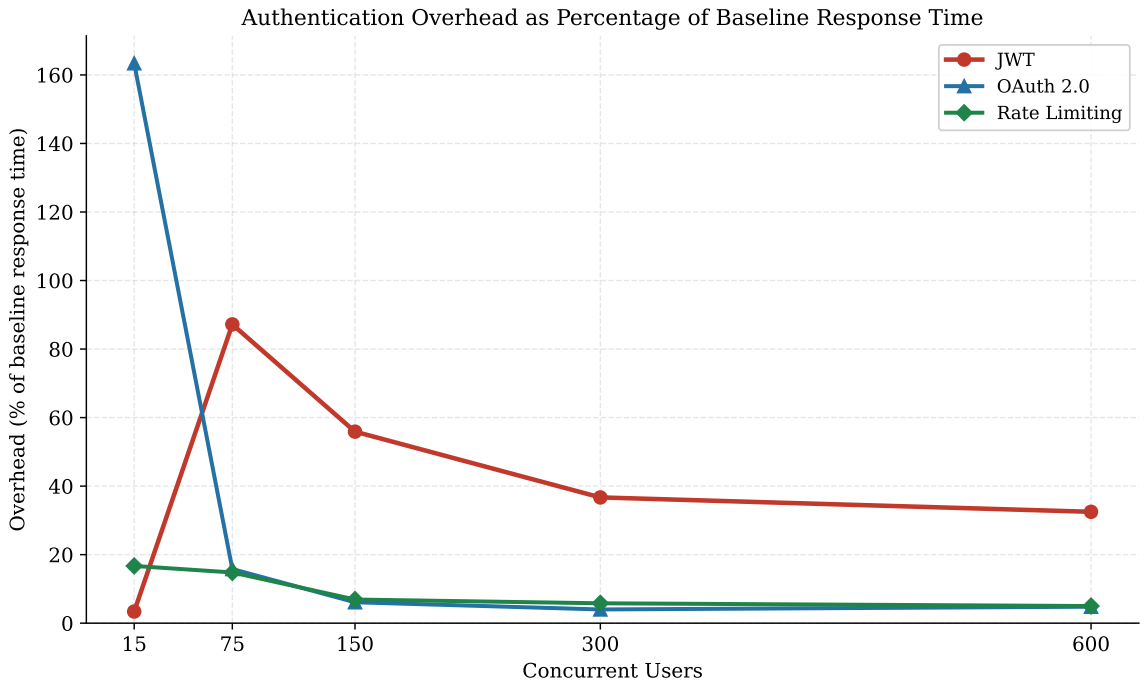


Figure D.3: Authentication Overhead as Percentage of Baseline Response Time. OAuth2's high relative overhead at low load (163.4%) reflects the small baseline denominator (3.12 ms); its absolute overhead (5.10 ms) remains within the 10 ms threshold.

Bibliography

[1] Dragoni, N., Giallorenzo, S., Lafuente, A. L., Mazzara, M., Montesi, F., Mustafin, R., & Safina, L. (2017). Microservices: Yesterday, today, and tomorrow. In *Present and Ulterior Software Engineering* (pp. 195–216). Springer. https://doi.org/10.1007/978-3-319-67425-4_12

[2] Blinowski, G., Ojdowska, A., & Przybyłek, A. (2022). Monolithic vs. microservice architecture: A performance and scalability evaluation. *IEEE Access*, 10, 20357–20374. https://doi.org/10.1109/ACCESS.2022.3152803

[3] Eling, M., & Wirfs, J. (2019). What are the actual costs of cyber risk events? *European Journal of Operational Research*, 272(3), 1109–1119. https://doi.org/10.1016/j.ejor.2018.07.021

[4] European Union Agency for Cybersecurity (ENISA). (2021). *Cybersecurity for SMEs: Challenges and Recommendations*. ENISA. https://www.enisa.europa.eu/publications/enisa-report-cybersecurity-for-smes

[5] Chidukwani, A., Zander, S., & Joiner, K. (2022). A survey on the cyber security of small-to-medium businesses. *IEEE Access*, 10, 85701–85719. https://doi.org/10.1109/ACCESS.2022.3197899

[6] OWASP Foundation. (2023). OWASP API Security Top 10 – 2023. https://owasp.org/API-Security/editions/2023/en/0x11-t10/

[7] Chatterjee, A., & Prinz, A. (2022). Applying Spring Security framework with KeyCloak-based OAuth2 to protect microservice architecture APIs. *Sensors*, 22(5), 1703. https://doi.org/10.3390/s22051703

[8] Yarygina, T., & Bagge, A. H. (2018). Overcoming security challenges in microservice architectures. In *2018 IEEE SOSE* (pp. 11–20). IEEE. https://doi.org/10.1109/SOSE.2018.00011

[9] Siriwardena, P. (2020). *Advanced API Security: OAuth 2.0 and Beyond* (2nd ed.). Apress. https://doi.org/10.1007/978-1-4842-2050-4

[10] de Almeida, M. G., & Canedo, E. D. (2022). Authentication and authorization in microservices architecture: A systematic literature review. *Applied Sciences*, 12(6), 3023. https://doi.org/10.3390/app12063023

[11] Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). *RFC 7519*, IETF. https://datatracker.ietf.org/doc/html/rfc7519

[12] Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson.

[13] Hardt, D. (Ed.). (2012). The OAuth 2.0 Authorization Framework. *RFC 6749*, IETF. https://datatracker.ietf.org/doc/html/rfc6749

[14] Barlette, Y., & Fomin, V. V. (2008). Exploring the suitability of IS security management standards for SMEs. In *Proceedings of HICSS*. https://doi.org/10.1109/HICSS.2008.167

[15] European Commission. (2003). Commission Recommendation of 6 May 2003 concerning the definition of micro, small and medium-sized enterprises. *Official Journal of the European Union*, L 124, 36–41.

[16] Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer. https://doi.org/10.1007/978-3-642-29044-2

[17] Bernstein, D. (2014). Containers and cloud: From LXC to Docker to Kubernetes. *IEEE Cloud Computing*, 1(3), 81–84. https://doi.org/10.1109/MCC.2014.51

[18] Felter, W., Ferreira, A., Rajamony, R., & Rubio, J. (2015). An updated performance comparison of virtual machines and Linux containers. In *2015 IEEE ISPASS* (pp. 171–172). https://doi.org/10.1109/ISPASS.2015.7095802

[19] Wolter, K., & Reinecke, P. (2010). Performance and security tradeoff. In *Formal Methods for Quantitative Aspects of Programming Languages* (pp. 135–167). Springer. https://doi.org/10.1007/978-3-642-13678-8_4

[20] Webster, J., & Watson, R. T. (2002). Analyzing the past to prepare for the future: Writing a literature review. *MIS Quarterly*, 26(2), xiii–xxiii.

[21] Kitchenham, B., Brereton, O. P., Budgen, D., Turner, M., Bailey, J., & Linkman, S. (2009). Systematic literature reviews in software engineering, A systematic literature review. *Information and Software Technology*, 51(1), 7–15. https://doi.org/10.1016/j.infsof.2008.09.009

[22] Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine].

[23] Ometov, A., Bezzateev, S., Mäkitalo, N., Andreev, S., Mikkonen, T., & Koucheryavy, Y. (2018). Multi-factor authentication: A survey. *Cryptography*, 2(1), 1. https://doi.org/10.3390/cryptography2010001

[24] Jain, R. (1991). *The Art of Computer Systems Performance Analysis*. Wiley-Interscience.

[25] Smith, C. U., & Williams, L. G. (2003). Software performance engineering. In *UML for Real* (pp. 343–365). Springer. https://doi.org/10.1007/0-306-48738-1_16

[26] Welsh, J. A., & White, J. F. (1981). A small business is not a little big business. *Harvard Business Review*, 59(4), 18–32.

[27] Caldwell, T. (2015). Securing small businesses – the weakest link in a supply chain? *Computer Fraud & Security*, 2015(9), 5–10. https://doi.org/10.1016/S1361-3723(15)30083-X

[28] Villamizar, M., Garcés, O., Castro, H., Verano, M., Salamanca, L., Casallas, R., & Gil, S. (2015). Evaluating the monolithic and the microservice architecture patterns. In *2015 10th Colombian Computing Congress*. IEEE. https://doi.org/10.1109/ColumbianCC.2015.7333476

[29] Pahl, C., & Jamshidi, P. (2016). Microservices: A systematic mapping study. In *Proceedings of CLOSER* (pp. 137–146). https://doi.org/10.5220/0005785501370146

[30] Berardi, D., Giallorenzo, S., Mauro, J., Melis, A., Montesi, F., & Prandini, M. (2022). Microservice security: A systematic literature review. *PeerJ Computer Science*, 8, e779. https://doi.org/10.7717/peerj-cs.779

[31] Sun, R., Wang, Q., & Guo, L. (2022). Research towards key issues of API security. In *Cyber Security: CNCERT 2021* (Vol. 1506, pp. 179–192). Springer. https://doi.org/10.1007/978-981-16-9229-1_11

[32] Akamai. (2024). *API Impact Security Study: How API Incidents Affect You and Your Team*. Akamai Technologies.

[33] Salt Security. (2024). *State of API Security Report, Q1 2024*.

[34] Jiang, Z. M., & Hassan, A. E. (2015). A survey on load testing of large-scale software systems. *IEEE Transactions on Software Engineering*, 41(11), 1091–1118. https://doi.org/10.1109/TSE.2015.2445340

[35] Anderson, R. (2020). *Security Engineering: A Guide to Building Dependable Distributed Systems* (3rd ed.). Wiley. https://www.cl.cam.ac.uk/~rja14/book.html

[36] Little, J. D. C. (1961). A proof for the queuing formula: $L = \lambda W$. *Operations Research*, 9(3), 383–387. https://doi.org/10.1287/opre.9.3.383

[37] Carias, J. F., Borges, M. R. S., Labaka, L., Arrizabalaga, S., & Hernantes, J. (2020). Systematic approach to cyber resilience operationalization in SMEs. *IEEE Access*, 8, 174200–174221. https://doi.org/10.1109/ACCESS.2020.3026063

[38] Gregor, S. (2006). The nature of theory in information systems. *MIS Quarterly*, 30(3), 611–642. https://doi.org/10.2307/25148742

[39] Barney, J. (1991). Firm resources and sustained competitive advantage. *Journal of Management*, 17(1), 99–120. https://doi.org/10.1177/014920639101700108

[40] Simon, H. A. (1955). A behavioral model of rational choice. *The Quarterly Journal of Economics*, 69(1), 99–118. https://doi.org/10.2307/1884852

[41] Simon, H. A. (1956). Rational choice and the structure of the environment. *Psychological Review*, 63(2), 129–138. https://doi.org/10.1037/h0042769

[42] Goldratt, E. M., & Cox, J. (1984). *The Goal*. North River Press.

[43] Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131–164. https://doi.org/10.1007/s10664-008-9102-8

[44] Glass, G. V., & Hopkins, K. D. (1996). *Statistical Methods in Education and Psychology* (3rd ed.). Allyn & Bacon.

[45] Nielsen, J. (1993). *Usability Engineering*. Academic Press.

[46] Card, S. K., Moran, T. P., & Newell, A. (1983). *The Psychology of Human-Computer Interaction*. Lawrence Erlbaum Associates.

[47] Dabrowski, J. R., & Munson, E. V. (2011). 40 years of searching for the best computer system response time. *Interacting with Computers*, 23(5), 555–564. https://doi.org/10.1016/j.intcom.2011.05.008

[48] Blanca, M. J., Alarcón, R., Arnau, J., Bono, R., & Bendayan, R. (2017). Non-normal data: Is ANOVA still a valid option? *Psicothema*, 29(4), 552–557. https://doi.org/10.7334/psicothema2016.383

[49] Cohen, J. (1988). *Statistical Power Analysis for the Behavioral Sciences* (2nd ed.). Lawrence Erlbaum Associates.

[50] Goldratt, E. M. (1990). *Theory of Constraints*. North River Press.

[51] Pereira, R., Couto, M., Ribeiro, F., Rua, R., Cunha, J., Fernandes, J. P., & Saraiva, J. (2017). Energy efficiency across programming languages: How do energy, time, and memory relate? In *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE 2017)* (pp. 256–267). ACM. `https://doi.org/10.1145/3136014.3136031`